AI Agent Araştırması

1. Klasik AI Agent Türleri

- Simple Reflex Agent
- Model-Based Reflex Agent
- Goal-Based Agent
- Utility-Based Agent
- Learning Agent

2. Modern LLM Agent Türleri

- Tool-Using Agent
- ReAct Agent
- Planning Agent
- Autonomous Agent
- Retrieval Agent
- Coding Agent
- Browser Agent
- Computer Use Agent
- Voice Agent
- Embodied Agent

3. Multi-Agent Mimarileri

- Multi-Agent System
- Hierarchical Agent
- Collaborative Agent
- Supervisor Pattern
- Manager–Worker
- Handoffs
- Agents as Tools

4.Agent Workflow (Design Patterns), Frameworkler ve Ekosistem Kavramları

1. Workflow Nedir?
2. Prompt Chaining
3. Routing
4. Parallelization
5. Orchestrator–Workers
6. Evaluator–Optimizer
7. Agent Framework'ler
8. Agent Ekosistemindeki Güncel Kavramlar

1. Klasik AI Agent Türleri

1. Simple Reflex Agent (Basit Refleks Ajanı)

Nedir? En temel yapay zekâ ajanı. Çevresinden aldığı anlık bilgiyi (current percept) değerlendirir, önceden tanımlı IF–THEN (Koşul–Eylem) kurallarına göre hemen aksiyon alır. Geçmişi hatırlamaz, geleceği planlamaz, öğrenemez — sadece mevcut çevreyi dikkate alır.

Nasıl çalışır?

Environment → Sensors → Current Percept → IF–THEN Rule → Action → Actuator
Süreç: çevrede olay olur → sensörler algılar → agent veriyi okur → IF–THEN kuralını uygular → hemen aksiyona geçer. Agent çevreyi *anlamaz*, sadece kural çalıştırır.

Gerçek hayattan örnekler:

- Robot süpürge: `Dust Sensor = HIGH` → `IF Dust > Threshold → Start Cleaning` (neden kirli olduğunu bilmez)
- Termostat: `Temperature = 18°C` → `IF Temperature < 20°C → Turn Heater On`
- Hareket sensörlü lamba: `IF Motion Detected → Turn Light On`
- Fabrika makinesi: `IF Machine Temperature > 100°C → Emergency Stop` (neden ısındığını araştırmaz)
- Üretim bandı: `IF Product Weight < Threshold → Reject Product` (neden hafif olduğunu analiz etmez)

Avantajları:

1. Çok hızlı — karar için hesaplama yapmaz, kurala bakıp uygular
2. Düşük kaynak kullanır — bellek kullanmaz, yoğun işlem gücü gerektirmez
3. Deterministik — aynı girişe her zaman aynı çıktı, güvenilir
4. Gerçek zamanlı — milisaniyede tepki verir, güvenlik sistemlerinde yaygın
5. Düşük maliyetli — ML, LLM, GPU, büyük veri gerektirmez

Dezavantajları:

- Hafızası yok — ne yaptığını bilmez (aynı noktayı tekrar temizleyebilir)
- İç dünya modeli yok — çevrenin haritasını çıkaramaz, daha önce gittiği yeri hatırlamaz
- Öğrenemez — yeni durumda kendi başına çözüm üretemez, insan yeni IF–THEN kuralı yazmalı
- Planlama yapamaz — uzun vadeli hedefleri değerlendiremez, sadece anlık tepki verir
- Belirsizlikle baş edemez — sensör yanlış veri gönderirse anlayamaz, yanlış karar verebilir

Günümüzde kullanım: Tek başına artık yaygın değil ama hızlı güvenlik katmanı olarak hâlâ kullanılıyor. Örnek — fabrikada: Simple Reflex (100°C'yi geçince hemen durdurur) +

Model-Based (normal ısınma mı arıza mı değerlendirir) + Utility-Based (durdurmak yerine hız düşürmek daha verimli mi hesaplar) birlikte çalışır.

AI Engineer Notu: Yalnızca mevcut çevreyi algılayarak önceden tanımlı IF–THEN kurallarına göre karar veren; hafızası, iç dünya modeli, öğrenme ve planlama yeteneği olmayan en temel yapay zekâ ajanı.

→ Neden bir sonraki geliştirildi: Sadece mevcut durumu bilir. Robot süpürge duvara çarpar → sağa döner → tekrar çarpar → tekrar döner... çünkü daha önce nereye gittiğini bilmiyor, odanın haritasını bilmiyor, hafızası yok.

2. Model-Based Reflex Agent (Model Tabanlı Refleks Ajanı)

Nedir? Simple Reflex'in geliştirilmişi. Anlık bilginin yanı sıra içsel bir dünya modeli (internal model) tutar. Bu sayede geçmiş algılarını hatırlar, çevre hakkında çıkarım yapar, kararını sadece o anki duruma değil önceki bilgilere de dayandırır.

Neden geliştirildi? Simple Reflex sadece o anı biliyordu — robot süpürge bir odayı temizledikten sonra hatırlayamaz, tekrar gidebilirdi. Model-Based bunun yerine: Salon ✔ / Mutfak ✖ / Yatak odası ✖ şeklinde takip eder, aynı yeri tekrar temizlemez.

Nasıl çalışır?

Environment → Sensors → Current Perception → Internal Model → Reasoning → Action → Actuator
Süreç: çevre algılanır → bilgi iç modele eklenir/güncellenir → geçmiş bilgi dikkate alınır → en uygun aksiyon seçilir → aktüatör uygular → iç model tekrar güncellenir.

4 temel bileşen :

1. Sensors — kamera, lidar, sıcaklık sensörü, API, GPS
2. Internal Model — geçmiş bilgiler, görünmeyen durumlar, mevcut çevre durumu saklanır, sürekli güncellenir
3. Reasoning Component — iç modele bakarak IF–THEN mantığıyla, artık geçmiş bilgiyi de hesaba katarak karar verir
4. Actuator — robot motorları, klima, fren sistemi, API çağrısı

Çalışma döngüsü (IBM, 4 aşama): `Sensing → Internal Modeling → Decision Making → Action` — her aksiyondan sonra yeniden başlar.

Gerçek hayattan örnekler:

- Robot süpürge: kir gördü → temizledi → haritaya kaydetti → bir daha aynı yere gitmedi
- Otonom araç: önündeki arabayı değil, bulunduğu şeridi, daha önce geçtiği yolu, trafik akışını da iç modelde tutar

- Akıllı ev: klima sadece sıcaklığı okumaz — "biraz önce açıldı, evde insan var, sıcaklık düşüyor" bilgisini de bilir

Avantajları:

- Geçmişi hatırlar, önceki bilgiyi kullanabilir
- Kısmen gözlemlenebilir ortamlarda çalışabilir — eksik bilgiyi iç model tamamlar
- Daha doğru kararlar verir — sadece anlık veriye göre hareket etmez
- Dinamik ortamlara daha uygun — değişiklikleri modele yansıtır

Dezavantajları:

- Daha fazla hesaplama gücü gerekir — iç modeli güncellemek maliyetli
- İç model yanlışsa kararlar da yanlış olur
- Öğrenemez — iç modeli günceller ama kuralları kendi başına değiştiremez, yeniden programlanması gerekir
- Uzun vadeli plan yapamaz — hâlâ reaktif, Goal-Based gibi hedef belirleyemez
- Tek bir göreve odaklıdır — birden fazla hedef arasında öncelik belirleyemez

Kullanım alanları: robot süpürgeler, otonom araçlar, robotik sistemler, akıllı ev otomasyonu, oyun AI'ları, üretim otomasyonu, depo robotları

AI Engineer Notu: Çevrenin içsel bir modelini oluşturarak geçmiş algıları ve çevre bilgisini saklayan; kararını sadece mevcut duruma değil bu iç modele de dayandıran, ama öğrenme ve uzun vadeli planlama yeteneği olmayan agent.

→ Neden bir sonraki geliştirildi: Geçmişi hatırlar ama "benim hedefim ne?" sorusunu soramaz. Robot sadece çevreyi anlamakla yetinmez; belirli bir noktaya gitmek, görevi tamamlamak ister.

3. Goal-Based Agent (Hedef Tabanlı Ajan)

Nedir? Belirli bir hedefe (goal) ulaşmak için çalışır. "Hedefime ulaşmak için hangi adımları izlemeliyim?" sorusunu sorar — artık tepki vermek yerine planlama (planning) yapar.

Neden geliştirildi? Model-Based geçmişi hatırlayabiliyordu ama "ben neyi başarmaya çalışıyorum?" bilmiyordu. Robot süpürge odanın haritasını bilebilir ama hangi odadan başlaması gerektiğini, hangi rotanın kısa olduğunu, önce mutfağı mı salonu mu temizlemesi gerektiğini hesaplayamaz.

Nasıl çalışır? (IBM, 4 aşama)

Goal Definition → Planning → Action Selection → Execution

Detaylı akış:

Environment → Sensors → Internal Model → Goal → Planning → Best Action → Actuator

(Model-Based'e ek olarak Goal + Planning katmanı eklenmiş)

4 aşama:

1. Goal Definition — agent'a hedef verilir ("Depodaki A paketini teslim et", "Robot mutfağı temizlesin")
2. Planning — adım adım plan kurulur (koridora git → mutfağa gir → masanın altını temizle → çık), rastgele hareket etmez
3. Action Selection — plana göre en uygun hareket seçilir (sağa mı sola mı)
4. Execution — plan uygulanır; ortam değişirse yeniden planlanır. IBM'in Google Maps örneği: önce rota oluşturur, trafik çıkarsa yeniden hesaplar.

Gerçek hayattan örnekler:

- Robot süpürge: amaç "tüm evi temizle" → en kısa rotayı hesapla → odaları sırayla temizle → şarj azalırsa istasyona dön → kaldığı yerden devam et
- Depo robotu (IBM): A rafındaki paketi kargoya götür → en kısa yolu hesaplar, engelleri dikkate alır, gerekirse rotayı yeniler
- Google Maps: İstanbul Havalimanı'na git → rota oluşturur, trafik olursa yeniden hesaplar, hedefe ulaştırır

Avantajları:

- Hedefe odaklı — ne yapacağını bilir
- Planlama yapabilir — önce plan, sonra uygulama
- Esnek — plan bozulursa yenisini kurar
- Karmaşık görevleri alt görevlere bölebilir

Dezavantajları:

- En iyi seçeneği hesaplayamaz — en ekonomik/güvenli/hızlı kararı veremez
- Trade-off yapamaz (hız mı, güvenlik mi, maliyet mi — seçemez)
- Öğrenemez — kuralları sabit kalır, yeni deneyimden ders çıkaramaz

Kullanım alanları: robotik, depo otomasyonu, navigasyon, otonom robotlar, workflow yönetimi, oyun AI'ları

AI Engineer Notu: Belirli bir hedefe ulaşmak için plan yapan, olası gelecek durumları değerlendirip uygun aksiyonu seçen; ama fayda (utility) hesaplaması ve öğrenme yeteneği olmayan agent.

→ Neden bir sonraki geliştirildi: Hedef belirleyebiliyor ama "birden fazla doğru yol varsa hangisi daha iyi?" sorusuna cevap veremiyor. Otonom araç için: en hızlı mı, en güvenli mi, en ucuz mu, en az yakıt tüketen mi rota? Goal-Based sadece hedefe ulaşmayı düşünür.

4. Utility-Based Agent (Fayda Tabanlı Ajan)

Nedir? Hedefe ulaşmaya çalışırken en faydalı (optimal) seçeneği seçer. "Hedefe ulaşmanın en iyi yolu hangisi?" sorusunu sorar — "en iyi" utility (fayda) fonksiyonuyla hesaplanır: her seçeneğe sayısal puan verilir, en yüksek puanlı seçilir.

Neden geliştirildi? Goal-Based hedefe ulaşabiliyordu ama birden fazla doğru çözüm olduğunda seçim yapamıyordu. Örnek: İstanbul→Ankara, 3 rota — Yol A (5 saat), Yol B (6 saat ama güvenli), Yol C (4 saat ama ücretli). Goal-Based: "hangisi olursa olsun ulaşırsam yeter" der. Utility-Based: "hangisi daha güvenli, ucuz, hızlı?" diye hesap yapar.

Utility Function nedir? Agent'ın en önemli bileşeni. Her seçeneğe bir fayda puanı (utility score) verir — örn. 91 puan alan Yol B seçilir. IBM'in tanımıyla: agent'ın değer sistemi gibidir; güvenlik, maliyet, verimlilik, kaynak kullanımı gibi kriterleri sayısal değerlendirir.

Nasıl çalışır? (IBM)

Perception → Internal Model → Action Generation → Outcome Prediction → Utility Assessment → Action Selection → Action

1. Perception — sensörlerden veri alınır
2. Internal Model — çevrenin güncel durumu oluşturulur
3. Action Generation — olası tüm hareketler belirlenir (sola git / sağa git / bekle)
4. Outcome Prediction — her hareketin olası sonucu tahmin edilir ("sola gidersem 5 dk kazanırım", "sağa gidersem yakıt tasarrufu")
5. Utility Assessment — her sonuca utility function uygulanır, puanlanır
6. Action Selection — en yüksek puanlı hareket seçilir
7. Action — karar uygulanır

Temel bileşenler (IBM): Utility Function, Sensors, Internal Model, Action Selection Mechanism, Actuators

Gerçek hayattan örnekler:

- Otonom araç: güvenlik, trafik, yakıt, süre, hava durumunu aynı anda değerlendirip utility function ile en uygun rotayı seçer
- Akıllı ev: klima elektrik maliyeti, kullanıcı konforu, enerji tasarrufu arasında denge kurar
- E-ticaret: öneri sistemi kullanıcı ilgisi, ürün fiyatı, stok durumu, kampanyaları birlikte değerlendirip önerir
- Dinamik fiyatlandırma: talep, rekabet, stok, mevsim bilgisiyle fiyat belirler

Avantajları:

- Birden fazla kriteri aynı anda değerlendirir — sadece hedefe değil en iyi sonuca odaklanır
- Belirsizlikle daha iyi baş eder — olasılıkları değerlendirebilir
- Esnek — farklı senaryoları karşılaştırabilir
- Daha kaliteli kararlar — trade-off yapabilir (hız/güvenlik/maliyet dengesi)

Dezavantajları:

- Utility function yazmak zor — agent'ın değer sistemi matematiksel tanımlanmalı
- Hesaplama maliyeti yüksek — her seçenek tek tek hesaplanır
- Öğrenemez — utility function kendiliğinden değişmez, insan müdahalesi gerekir

- Etik problemler — örn. otonom araç iki kötü seçenek arasında kaldığında hangi kriterin önceliklendirileceği utility function'a bağlı, bu yüzden etik tasarım önemli

Kullanım alanları: otonom araçlar, robotik, akıllı ev sistemleri, sağlık, tavsiye sistemleri, dinamik fiyatlandırma, lojistik ve tedarik zinciri

AI Engineer Notu: Birden fazla olası eylemi değerlendirip her birine utility (fayda) değeri atayan, en yüksek faydayı sağlayan seçeneği tercih eden agent.

→ Neden bir sonraki geliştirildi: En faydalı seçeneği seçebiliyor ama geçmiş deneyimlerden öğrenemez. Örn. bir tavsiye sistemi yanlış öneri yaptığında kendi kendine düzeltemez, aynı utility function'ı kullanmaya devam eder. Bu yüzden **Learning Agent** geliştirildi — geri bildirim ve deneyimlerden öğrenerek zamanla davranışını iyileştirir, yeni stratejiler geliştirir.

5. Learning Agent (Öğrenen Ajan)

Nedir? Deneyimlerinden öğrenebilen ve zamanla performansını geliştiren agent. Diğerlerinden farkı: yalnızca verilen kuralları uygulamaz — işlemlerin sonucunu değerlendirir, hatalarından ders çıkarır, gelecekte daha doğru kararlar vermeye çalışır. Her görevden sonra kendini geliştiren agent.

Neden kullanılır? Bazı problemler sürekli değişir, tüm kuralları önceden belirlemek mümkün değildir. Learning Agent geçmiş deneyimlerden öğrenir, başarılı/başarısız sonuçları analiz eder, stratejisini günceller, zamanla daha iyi kararlar verir — sürekli insan müdahalesi olmadan kendini geliştirebilir.

Nasıl çalışır? 4 temel bileşen:

1. Performance Element — mevcut bilgiyi kullanarak görevi yerine getirir
2. Learning Element — sonuçları analiz eder, yeni bilgi öğrenir
3. Critic — üretilen sonucu değerlendirip geri bildirim sağlar
4. Problem Generator — daha fazla öğrenmek için yeni deneyimler/farklı çözümler dener

Akış:

Çevre → Agent Görevi Yapar → Sonuç Oluşur → Critic Değerlendirir → Learning Element Öğrenir → Davranış Güncellenir → Bir Sonraki Görevde Daha İyi Performans

Gerçek hayattan örnekler:

- E-ticaret öneri sistemi: başta rastgele ürün önerir → tıklama/satın alma/görmezden gelme verisini analiz eder → zamanla daha isabetli öneriler sunar
- Otonom araçlar: sürüş sırasında karşılaşılan durumları analiz ederek zamanla daha güvenli sürüş stratejileri geliştirir

Avantajları:

- Deneyimlerinden öğrenebilir
- Zamanla performansını artırır
- Değişen ortamlara uyum sağlar
- Sürekli insan müdahalesi ihtiyacını azaltır

Dezavantajları:

- Öğrenme süreci zaman alabilir
- Kaliteli geri bildirim ve veri gerektirir
- Yanlış/eksik verilerden hatalı öğrenebilir
- Eğitim ve hesaplama maliyeti diğer agent türlerine göre daha yüksek

AI Engineer Notu: Yaptığı işlemlerden elde ettiği geri bildirimi kullanarak davranışını geliştiren ve zamanla daha başarılı hale gelen agent. Performansını sürekli iyileştirebildiği için öneri sistemleri, otonom araçlar, robotik ve kişiselleştirilmiş AI uygulamalarında yaygın kullanılır.

Klasik Agent Evrimi:

Simple Reflex → Model-Based Reflex → Goal-Based → Utility-Based → Learning Agent
→ Her yeni agent, bir öncekinin eksik kaldığı yeteneği tamamlar: hafıza (Model-Based) → hedef/planlama (Goal-Based) → optimizasyon/trade-off (Utility-Based) → öğrenme/kendini geliştirme (Learning Agent).

2. Modern LLM Agent Türleri

- Tool-Using Agent
- ReAct Agent
- Planning Agent
- Autonomous Agent
- Retrieval Agent
- Coding Agent
- Browser Agent
- Computer Use Agent
- Voice Agent
- Embodied Agent

1. Tool-Using Agent (Tool Calling)

Neden ortaya çıktı? LLM'ler eğitim verisiyle sınırlı — güncel hava durumu, döviz kuru, e-posta, takvim, SQL sorgusu gibi şeyleri kendi başına bilemez/yapamaz. Çözüm: "bilmediğin işi başka programa yaptır" → Tool Calling.

Nedir? Kendi başına yapamadığı işleri dış araçlarla yapan agent. Artık sadece cevap vermez, iş de yapar.

Eski: Kullanıcı → LLM → Cevap
Yeni: Kullanıcı → LLM → Tool gerekli mi? → Evet → Tool çalıştır → Sonucu al → Cevap ver
Eskiden LLM sadece Training Data ile sınırlıydı; şimdi + API + Database + Python + Search + Calendar + Email + RAG + MCP kullanabiliyor. LLM'in beyni büyümedi, sadece eline alet verildi.

Neden gerekli? LLM internete çıkamaz, database okuyamaz, mail gönderemez, dosya açamaz, Python çalıştıramaz, takvim oluşturamaz — ama Tool kullanırsa hepsini yapabilir.

Nasıl çalışır (6 adım):

1. Kullanıcı isteği — LLM okur
2. LLM karar verir — "bunu biliyor muyum? hayır → Weather API kullanmalıyım" (en önemli kısım)
3. Doğru Tool'u seçer — sistemde Weather/Calendar/Email/Database/Python Tool varsa uygun olanı seçer
4. Tool'a istek gönderir — `{"city":"Istanbul"}` gibi formatlı istek; Tool'ların adı, açıklaması, parametreleri bu yüzden çok önemli
5. Tool sonucu döndürür — `{"temperature":32,"humidity":48}`
6. LLM sonucu insana çevirir — "İstanbul'da hava şu anda 32°C ve nem oranı %48"

Tool türleri:

1. Information Retrieval — Google, Wikipedia, RAG, Database, Search
2. Code Execution — Python, Matlab, Wolfram Alpha (örn. "100 milyon satırlık csv analiz et" → LLM kendi yapmaz, Python Tool kullanır)
3. Process Automation — mail gönder, takvim oluştur, Slack mesajı at, CRM güncelle
4. Smart Devices — IoT, robot, akıllı ev, drone, endüstriyel cihaz

Tool Calling vs RAG: Her RAG bir Tool olarak kullanılabilir, ama her Tool RAG değildir.

LangChain neden geçiyor? Tool'ları yönetir, parametreleri hazırlar, gerekirse birden fazla Tool'u sırayla çalıştırır, önceki Tool sonuçlarını hafızada tutar — Tool Calling'i organize eden framework.

Örnek: Flight AI (Kullanıcı → LLM → Tool Call → Python Function → SQLite → Fiyat → LLM → Cevap), MCP Leave Manager (Claude → Leave Request Tool → SQLite → izin bilgisi → Claude), PDF RAG'de Retriever de aslında bir Tool gibi çalışıyor.

Akış özeti:

Kullanıcı isteğini analiz eder → Tool gerekli mi karar verir → Doğru Tool'u seçer → Parametreleri oluşturur → Tool'u çalıştırır → Sonucu alır → LLM doğal dile çevirip sunar

2. ReAct Agent (Reasoning + Acting)

Nedir? LLM'in adım adım düşünmesini (Reasoning) ve gerektiğinde dış araç kullanarak harekete geçmesini (Acting) sağlayan agent mimarisi. 2023'te Yao ve arkadaşlarının "ReAct: Synergizing Reasoning and Acting in Language Models" makalesiyle tanıtıldı; LangGraph, LlamaIndex gibi birçok framework'ün temelini oluşturur.

Neden ortaya çıktı? İlk LLM'ler Soru → LLM → Cevap şeklinde çalışıyordu; bilgi eksikse veya dış işlem gerekiyorsa başarısız oluyordu. Örn. "Bugünkü hava durumuna göre kıyafet öner" — normal LLM havayı bilemez. ReAct önce düşünür, sonra bilgi toplar, tekrar düşünür.

Think–Act–Observe Loop:

Thought → Action → Observation → Thought → Action → Observation → Final Answer

1. Thought — "kullanıcı hava durumu istiyor → gerçek zamanlı veri gerekli → Weather API kullanmalıyım"
2. Action — Weather API, city=Istanbul çalıştırılır
3. Observation — Temperature=31°C, Rain=Yes sonucu alınır, bağlama eklenir
4. Tekrar Thought — "yağmur var → şemsiye önermeliyim"
5. Final Answer — "İstanbul'da bugün yağmur bekleniyor, şemsiye alın"

İnsan gibi düşünme benzetmesi: seyahate çıkarken zihinde "hava nasıl olacak? → telefondan bak → soğukmuş → dolabı kontrol et → kalın mont al" süreci — ReAct aynı mantığı uygular.

ReAct Prompting: modele "önce düşün, gerekirse Tool kullan, sonucu değerlendir, gerekirse tekrar Tool kullan, sonunda cevap ver" istemi verilir. Yapı: `Question → Thought → Action → Action Input → Observation → Thought → ... → Final Answer`

LangGraph'ta ReAct: iki temel node — LLM Node ve Tool Node. LLM düşünür, Tool çalışır, sonuç tekrar LLM'e döner, gerekirse döngü yeniden başlar. LangGraph state (durum/hafıza) tutarak önceki adımların unutulmamasını sağlar.

Örnekler:

- Uçak bileti: Thought (uçuş bilgisi lazım) → Action (Flight API) → Observation (THY 12.000 TL, Pegasus 8.500 TL) → Thought (Pegasus ucuz) → Final Answer
- IT destek: "yeni destek kaydı oluştur" → düşün → create_ticket Tool → ticket oluşturuldu → kullanıcıya bilgi ver (Cloud Object Storage'daki dosyayı okuyup güncelleyip LangGraph üzerinden yönetiyor)

Avantajları: adım adım akıl yürütür, birden fazla Tool kullanabilir, yeni bilgiye göre planını değiştirebilir, dış kaynaktan bilgi aldığı için halüsinasyonu azaltır, düşünme süreci takip edilebildiği için hata ayıklaması kolaydır.

Dezavantajları: daha fazla token tüketir, daha yavaş çalışabilir, döngü sınırlandırılmazsa gereksiz tekrar oluşabilir, güçlü reasoning yeteneğine sahip LLM'lerle daha iyi çalışır.

AI Engineer Notu: LLM'in önce akıl yürütmesini (Reasoning), ardından gerekli Tool'ları kullanmasını (Acting) ve her Tool sonucunu değerlendirerek yeni kararlar almasını sağlayan, Thought → Action → Observation döngüsüyle çalışan modern agent mimarisi.

→ Neden bir sonraki geliştirildi: Her adımda düşünüp hareket eder, ama büyük/karmaşık görevlerde (örn. "7 günlük Japonya seyahati planla") önce genel plan oluşturmak sonra uygulamak daha verimlidir.

3. Planning Agent (Planlama Yapan Ajan)

Nedir? Hedefe ulaşmadan önce genel bir plan oluşturan, ardından bu planı adım adım uygulayan agent. Tool-Using doğrudan Tool kullanır, ReAct düşün-eylem-gözlem döngüsüyle ilerler; Planning Agent işe başlamadan önce "bu görevi nasıl yapmalıyım?" diye düşünüp strateji kurar.

Neden geliştirildi? ReAct küçük görevlerde başarılı ama büyük görevlerde sürekli yeniden düşünmek verimsiz. Örn. "10 günlük Japonya gezisi planla" — ReAct sürekli Düşün→Araştır→Düşün→Araştır döngüsüne girer; Planning Agent önce bütün işi planlar.

Akış:

Goal → Task Decomposition → State Representation → Action Sequencing → Optimization → Execution → Feedback → Replanning (gerekirse)

1. Goal Definition — hedef belirlenir ("Paris seyahati planla", "satış raporu hazırla")
2. Task Decomposition — büyük görev küçük görevlere bölünür (İtalya Gezisi → Uçak/Otel/Rota/Restoran/Bütçe)
3. State Representation — mevcut durum modellenir (robot için Konum/Engeller/Batarya/Harita; chatbot için Kullanıcı isteği/Geçmiş konuşma/Memory/RAG bilgisi)
4. Action Sequencing — hangi iş hangi sırada yapılacak belirlenir (Uçak al → Otel seç → Araba kirala → Gezi planı oluştur)
5. Optimization — birden fazla plan varsa en iyisi seçilir (Heuristic Search, Reinforcement Learning, Probabilistic Planning yöntemleriyle; en kısa/ucuz/güvenli/verimli plan seçilir)
6. Execution — plan uygulanır (Tool Calling, RAG, API, Function Calling devreye girer)
7. Replanning — ortam değişirse plan yeniden kurulur ("uçuş iptal oldu → yeni uçuş bul → oteli güncelle" veya "robotun önü kapandı → yeni rota hesapla")

Örnekler:

- Seyahat: "5 günlük Roma gezisi hazırla" → Goal → Uçak → Otel → Müze → Restoran → Bütçe → Takvim
- Robot: "kutuyu A'dan B'ye taşı" → en kısa rota → engeller → batarya → alternatif yol
- Depo robotu: Sipariş → en yakın raf → en uygun rota → teslimat

Avantajları: büyük görevleri yönetebilir, görevi küçük parçalara böler, daha organize çalışır, ortam değişirse planını günceller.

Dezavantajları: plan oluşturmak zaman alır, hesaplama maliyeti yüksek, çok dinamik ortamlarda sürekli yeniden planlama gerekebilir.

AI Engineer Notu: Hedefe ulaşmak için görevi alt parçalara ayıran, aralarında yürütme planı oluşturan, planı uygulayan ve gerekirse çevredeki değişikliklere göre yeniden planlayan modern agent türü.

Örnek: DealAgentFramework'te hangi agent'ın çalışacağı planlanıyordu, MCP Leave Manager'da hangi tool'ların çağrılacağı planlanıyordu; Flight AI akışı basitti, tam bir Planning Agent değildi. Günümüzde Deep Research / Claude Research sistemleri büyük ölçüde Planning Agent mantığıyla çalışır: önce planla, sonra araştır, sonra üret.

Tool-Using Agent → ReAct Agent → Planning Agent → Autonomous Agent (sıradaki)

4. Autonomous Agent (Otonom Ajan)

Nedir? Kullanıcıdan yalnızca bir hedef aldıktan sonra bu hedefe ulaşmak için kararları kendi veren, plan yapan, öğrenen, araç kullanan ve gerektiğinde davranışını değiştirebilen agent. Minimum insan müdahalesiyle bağımsız çalışır.

Neden geliştirildi? Planning Agent plan oluşturabiliyordu ama çoğu zaman kullanıcı süreci yönetmeye devam ediyordu. Autonomous Agent: "sen bana sadece hedefi söyle, gerisini ben hallederim."

Akış:

Goal → Real-Time Data Collection → Data Processing → Decision Making → Planning → Tool Calling → Execution → Feedback → Learning & Adaptation

1. Gerçek zamanlı veri toplama — sensörler, API'ler, IoT cihazları, veritabanları, web servisleri
2. Veri işleme — toplanan veriler analiz edilir (hava durumu + trafik + takvim + kullanıcı tercihleri birleştirilir)
3. Karar verme — hangi aksiyonun doğru olduğuna kendisi karar verir (ML, Deep Learning, Reasoning birlikte kullanılabilir)
4. Planning — görevi küçük parçalara böler (sunum hazırla → veri topla → grafik oluştur → PowerPoint hazırla → mail gönder)
5. Tool Calling — Python, Web Search, SQL, Calendar, Email, MCP Server gibi araçları seçer
6. Execution — plan uygulanır, araçlar çalıştırılır
7. Feedback — sonuç kontrol edilir ("API başarısız → alternatif API kullan")
8. Learning & Adaptation — geçmiş deneyimden öğrenir, yeni durumlara uyum sağlar, zamanla daha iyi karar verir

Temel özellikler: Autonomous Decision Making (insan yerine karar verebilir), Continuous Learning, Adaptability, Enterprise Integration (CRM/ERP/Takvim/Mail/Veritabanı/MCP ile çalışabilir), Independent Execution.

Örnekler:

- Copilot: "haftalık satış raporunu hazırla" → Excel'i oku → grafikleri oluştur → PowerPoint hazırla → Outlook ile gönder
- Yazılım geliştirme: bug bul → kod oku → test çalıştır → hata düzelt → tekrar test et → PR oluştur
- Sağlık: hasta verilerini analiz et → riskleri belirle → doktoru uyar
- Üretim: sensörleri izle → arıza tahmini yap → bakım talebi oluştur

Avantajları: insan müdahalesini azaltır, büyük/karmaşık görevleri yönetebilir, gerçek zamanlı karar verebilir, sürekli öğrenebilir, yeni durumlara uyum sağlar.

Dezavantajları: güvenlik/gizlilik riskleri oluşturabilir, yanlış kararlar zincirleme hatalara yol açabilir, güçlü guardrails gerektirir, geliştirme/altyapı maliyeti yüksektir.

AI Engineer Notu: Kullanıcıdan yalnızca yüksek seviyeli hedef alan; planlama, karar verme, Tool Calling, yürütme, geri bildirim ve öğrenme süreçlerini büyük ölçüde kendi başına gerçekleştirerek görevi minimum insan müdahalesiyle tamamlayan modern agent türü.

Tool-Using → ReAct → Planning → Autonomous
Autonomous Agent bunların hepsini birleştirir: Tool kullanabilir, ReAct mantığıyla düşünebilir, plan oluşturabilir ve tüm bunları kullanıcı yönlendirmeden yürütebilir.

5. Retrieval Agent (RAG Agent)

Nedir? Cevap üretmeden önce ihtiyaç duyduğu bilgileri dış kaynaklardan (vektör veritabanı, PDF, web, SQL, API) bulan, bunu bağlam olarak LLM'e veren ve doğru cevabı üreten agent. Bu yapı RAG veya gelişmiş haliyle Agentic RAG olarak adlandırılır.

Neden geliştirildi? LLM'lerin iki problemi: (1) güncel bilgi eksikliği — "bugünkü dolar kuru kaç?" bilemez; (2) kurumsal bilgi eksikliği — "şirketin izin politikası nedir?" bilemez. Çözüm: Soru → Bilgiyi Bul (Retrieve) → LLM'e Ver → Cevap Üret.

Retriever vs Retrieval Agent — en önemli fark:

- Retriever: sadece chunk bulan bileşen. Soru → Qdrant → en benzer 5 chunk. Görevi bitti, asla cevap üretmez.
- Retrieval Agent: hangi kaynağa gidileceğine karar veren, gerekirse birden fazla Retriever/Tool kullanan akıllı agent. Retriever'ları yönetir.

Klasik RAG akışı:

Soru → Retriever → Vector DB → Chunk → LLM → Cevap
Tek bilgi kaynağı, genelde tek seferlik retrieval, statik.

Agentic RAG akışı:

Soru → Routing → Planning → PDF Retriever / Web Search / SQL / API → Bilgileri Birleştir → LLM → Cevap
Artık plan yapabilir, Tool kullanabilir, birden fazla veri kaynağını sorgulayabilir, sonuçları değerlendirip tekrar sorgu oluşturabilir.

Getirdiği yenilikler:

1. Flexibility — birden fazla bilgi kaynağından veri çekebilir, tek Vector DB ile sınırlı değil
2. Adaptability — duruma göre farklı strateji, gerekirse yeni sorgu oluşturur
3. Accuracy — bulduğu bilgiyi değerlendirir, gerekirse yeniden arama yapar
4. Scalability — birden fazla Agent birlikte çalışabilir, büyük kurumsal sistemlere uygun
5. Multimodality — metin dışında görsel, ses, tablo, PDF ile de çalışabilir

Agentic RAG içindeki agent türleri: Routing Agent (hangi kaynak kullanılacak), Query Planning Agent (soruyu alt sorgulara böler), ReAct Agent (araştırırken yeni kararlar verir), Plan-and-Execute Agent (planı oluşturur, alt görevleri yürütür).

Örnek: "Apple'ın 2025 satışları ile şirketimizin satışlarını karşılaştır" → Apple verisi internette, şirket verisi SQL'de, politika PDF'de → Web Search + SQL + PDF → birleştir → LLM → karşılaştırmalı rapor. Klasik RAG bunu tek başına yapamaz.

Örnek: PDF RAG (PDF → Chunk → Embedding → Qdrant → Retriever → LLM), InsureLLM (Markdown → ChromaDB → Retriever → LLM), Menü RAG — bunların hepsi klasik RAG'dı. Eğer Web Search, SQL, Planning, Routing, ReAct, Multi-Agent eklenseydi Agentic RAG olurdu.

Avantajları: güncel bilgi kullanır, halüsinasyonu azaltır, birden fazla veri kaynağı kullanabilir, kurumsal bilgiyle çalışabilir, fine-tuning gerektirmez, karmaşık bilgi toplama süreçlerini yönetebilir.

Dezavantajları: retriever yanlış bilgi bulursa cevap da yanlış olur, çok agent kullanınca maliyet artar, daha fazla token tüketebilir, tamamen halüsinasyonsuz değildir.

AI Engineer Notu: Cevap üretmeden önce ihtiyaç duyduğu bilgilerin hangi kaynakta olduğuna karar veren, gerekirse birden fazla Retriever ve Tool kullanarak PDF/Vector DB/SQL/Web/API'den bilgi toplayan, bunları birleştirip LLM'e bağlam olarak sunan modern agent türü.

6. Coding Agent

Nedir? Yazılım geliştirme görevlerini kullanıcı adına gerçekleştiren agent. Sadece kod önermez — projeyi inceler, dosyaları okur, kod yazar/düzenler, terminal komutları çalıştırır, testleri çalıştırır, hataları bulup düzeltir, yeni dosyalar oluşturur, sonucu kontrol eder.

Normal kod asistanından farkı:

Normal: Kullanıcı sorusu → Kod önerisi → Kullanıcı kendisi uygular
Coding Agent: Görev → Projeyi incele → Plan oluştur → Dosyaları oku → Kodu değiştir → Test çalıştır → Hata varsa düzelt → Görevi tamamla
Kod öneren chatbot değil, kod üzerinde gerçekten çalışan agent.

Genel döngü:

Kullanıcı görevi → LLM → Tool gerekli mi? → Dosya oku/Kod yaz/Shell çalıştır → Tool sonucu → LLM tekrar değerlendirir → Gerekirse yeni işlem → Görev tamamlanır

Temel yetenekleri:

1. Repository ve dosya analizi — proje yapısını anlar (app/, tests/, config/, requirements.txt), ilgili dosyayı belirler
2. Kod okuma — read_file gibi araçlarla hatanın olduğu fonksiyonu, test dosyasını, config'i, API route'unu okur
3. Kod yazma/düzenleme — yeni fonksiyon yazabilir, mevcut kodu değiştirebilir, dosya oluşturabilir, çoklu dosyada toplu değişiklik yapabilir (write_file, edit_files gibi araçlarla)
4. Terminal/Shell kullanımı — pytest, npm test, python app.py, git status, grep, docker compose up gibi komutları execute aracıyla çalıştırır
5. Test ve doğrulama — kodu değiştir → testleri çalıştır → hata varsa tekrar düzelt, yoksa tamamla
6. Plan oluşturma — büyük görevlerde önce plan hazırlar (örn. "1. Auth servisini incele 2. Null pointer kaynağını bul 3. Kontrol ekle 4. Test yaz 5. Tüm testleri çalıştır"); Plan Mode'da uygulama yapmadan önce kullanıcıya plan sunar
7. Sub-agent kullanımı — görevi alt agent'lara bölebilir (Root Coding Agent → Backend/Frontend/Infrastructure Agent), paralel çalışıp sonuçları ana agent'a gönderir (spawn_agent ile)

Örnek görev: "Auth servisindeki null pointer hatasını bul ve düzelt" → görevi analiz et → repository yapısını incele → auth dosyalarını bul → logları/testleri oku → hatanın kaynağını tespit et → kodu düzelt → testleri çalıştır → başarısızsa tekrar düzenle → sonucu açıkla

Workspace: agent'ın kodu okuyabildiği, dosya değiştirebildiği, terminal komutu çalıştırabildiği geliştirme ortamı (kaynak kod + terminal + testler + git). Sadece mimari sorulursa workspace gerekmez; kod okuma/değiştirme gerekiyorsa açılır.

Gerçek hayattaki örnekler: OpenAI Codex, Claude Code, GitHub Copilot coding agent, Cursor Agent, Coder Agents, Devin benzeri sistemler.

Avantajları: geliştirme süresini kısaltır, tekrarlayan görevleri otomatikleştirir, büyük kod tabanlarını inceleyebilir, testle sonucu doğrulayabilir, çoklu dosyada tutarlı değişiklik yapabilir, alt agent'larla paralel çalışabilir.

Dezavantajları: yanlış dosyaları değiştirebilir, hatalı/güvensiz kod yazabilir, terminal erişimi güvenlik riski oluşturur, büyük projelerde bağlamı kaçırabilir, insan incelemesi yine gereklidir, yetkiler sınırlandırılmazsa tehlikeli komutlar çalıştırabilir.

Güvenlik: kaynak kod, terminal, git, API anahtarları, veritabanları, ağ bağlantılarına erişebildiği için kritik. API anahtarlarının workspace içine konmaması, agent'ın yalnızca kullanıcıyla aynı yetkilere sahip olması, workspace erişiminin izole edilmesi önemli.

Farklar: Tool-Using Agent ile → Coding Agent, yazılım geliştirme araçlarıyla donatılmış uzman bir Tool-Using Agent olarak düşünülebilir. Autonomous Agent ile → Coding Agent aynı zamanda otonom olabilir (hatayı bulup düzeltip test etmesi istendiğinde kendi kararlarını verir).

AI Engineer Notu: Yazılım geliştirme görevini analiz eden, proje dosyalarını/kod tabanını inceleyen, kod yazma-düzenleme araçlarını kullanan, terminal komutları ve testler çalıştıran, sonuca göre çözümünü yineleyen uzmanlaşmış agent türü. Sadece kod üretmez; kod tabanında işlem yapar ve yaptığını test ederek doğrular.

7. Browser Agent

Nedir? Kullanıcı adına internet tarayıcısını kullanabilen, web sitelerini ziyaret eden, bilgi toplayan, formları dolduran, fiyat karşılaştıran ve web görevlerini otonom tamamlayabilen agent.

Neden ortaya çıktı? İnternet insanlar için tasarlanmıştı — bir bilgiye ulaşmak Google → 10 farklı site → karşılaştır → karar ver sürecini gerektiriyordu, zaman alıyordu. Browser Agent bu süreci otomatikleştirir.

Akış:

Kullanıcı İsteği → Planning → Browser Aç → Web Sitelerini Gezin → Bilgileri Topla → Karşılaştır → Sonucu Sun
Yeni sekme açabilir, sayfalar arası gezinebilir, butona tıklayabilir, form doldurabilir, arama yapabilir, sonuç analiz edebilir.

Temel yetenekleri:

1. Web Navigation — Google, Amazon, Booking, GitHub, LinkedIn gibi sitelerde gezinebilir
2. Web Search — "en ucuz Paris uçak bileti" gibi arama yapabilir
3. Data Extraction — fiyat, ürün, tarih, haber, tablo bilgisini okuyabilir
4. Form Filling — rezervasyon, üyelik, başvuru formu doldurabilir
5. Decision Making — Hotel A/B/C'yi karşılaştırıp en uygununu seçebilir
6. Multi-Step Tasks — birden fazla adımı kendi yönetebilir

Kullandığı teknolojiler: Planning, Tool Calling, bazı sistemlerde Computer Vision, OCR, DOM Analizi, Browser Automation, Memory.

Örnekler:

- "Berlin için en uygun uçuşu ve oteli bul" → Google Flights → Pegasus/THY → Booking/Hotels → fiyatları karşılaştır → listele
- "iPhone 18'in en uygun fiyatını bul" → Amazon → Hepsiburada → Trendyol → Apple Store → karşılaştır → sonucu göster

Avantajları: saatler süren araştırmayı dakikalara indirir, tekrarlayan web işlemlerini otomatikleştirir, çok siteyi aynı anda inceleyebilir, bilgileri karşılaştırabilir, dijital operasyonu kullanıcı adına gerçekleştirebilir.

Dezavantajları: web siteleri değişirse hata yapabilir, CAPTCHA/giriş doğrulamada zorlanabilir, gizlilik/güvenlik açısından dikkat gerekir, yanlış web bilgisi kullanabilir.

Gerçek hayatta kullanılan sistemler: Operator, Manus, Browser Use, Skyvern, Stagehand, BrowserBase.

Gelecekte önemi: eskiden internetin kullanıcısı insandı, yakın gelecekte önemli kullanıcılardan biri AI Agent'lar olacak — insanlar "araştır, karşılaştır, sonucu getir" diyecek, agent interneti dolaşıp getirecek. Bu yüzden SEO'nun yanında GEO (Generative Engine Optimization), AI uyumlu web siteleri, agent-friendly arayüzler giderek önem kazanıyor.

AI Engineer Notu: Web tarayıcısını kontrol ederek internette gezinebilen, web sitelerinden bilgi toplayabilen, karşılaştırma yapabilen, form doldurabilen ve çok adımlı web görevlerini otonom tamamlayabilen agent. En ayırt edici özelliği sadece bilgi okumaması, gerçek tarayıcı üzerinde insan gibi etkileşime girebilmesidir.

8. Computer Use Agent (CUA)

Nedir? Yalnızca web tarayıcısını değil, bilgisayarın ekranını, faresini ve klavyesini kullanarak kullanıcı adına dijital görevleri yerine getiren agent.

Neden ortaya çıktı? Browser Agent yalnızca web sitelerinde çalışıyordu ama insanlar Excel, Word, Photoshop, Slack, VS Code, SAP, CRM gibi masaüstü uygulamalarını da kullanıyor. CUA artık bilgisayarın tamamını kullanabiliyor.

Browser Agent vs CUA:

Browser Agent → Sadece Chrome içinde çalışır
CUA → Chrome, Excel, Word, VS Code, Slack, Dosya Gezgini — her yerde çalışabilir
Akış:

User Request → Screen Capture → Reasoning → Mouse/Keyboard Action → Yeni Ekran → Reasoning → Yeni Action → Task Complete

1. Perception — ekran görüntüsü alınır. Model HTML okumaz, API kullanmaz — direkt ekrandaki piksellere bakar, gerçekten insan gibi görür.
2. Reasoning — "Login butonu sağ üstte → önce ona tıklamalıyım → sonra e-mail yazmalıyım"
3. Action — mouse hareket ettir, sol tık, scroll, yazı yaz, enter bas — gerçek fare/klavye hareketleri
4. Observation — yeni ekran gelir, agent tekrar inceler, görev bitene kadar döngü devam eder

Browser Agent'tan en büyük farkı: Browser Agent DOM/HTML/web elementleriyle çalışır; CUA bunlara ihtiyaç duymaz, sadece ekran görüntüsüne bakar (Pixel → Reasoning → Mouse → Keyboard). API gerektirmez — programı API olmasa bile insan gibi ekrana bakarak kullanabilir.

Örnekler:

- Excel: "satış raporunu grafiğe çevir" → Excel'i aç → dosyayı bul → veriyi seç → Insert Chart → grafik oluştur
- Gmail: "Ayşe'ye raporu gönder" → Chrome aç → Gmail → Compose → mail yaz → dosya ekle → Send
- VS Code: "projeyi çalıştır" → VS Code → Terminal Aç → npm install → npm run dev

Güvenlik: şifre girme, CAPTCHA çözme, ödeme yapma gibi hassas işlemlerde kullanıcıdan onay istenir — yanlış işlemleri önlemek için eklenen katman.

Avantajları: API gerektirmez, her uygulamada çalışabilir, insan gibi bilgisayar kullanabilir, çok adımlı görevleri tamamlayabilir, hatalara göre kendini düzeltebilir.

Dezavantajları: görsel arayüz değişirse hata yapabilir, insan kadar başarılı değildir, fare/klavye ile çalıştığı için bazı görevlerde yavaştır, güvenlik açısından dikkatli kullanılmalıdır.

Kullanılan sistemler: Operator (CUA kullanır), Copilot Actions (benzer yaklaşım), Anthropic Computer Use, Browser Use, UI otomasyon sistemleri.

AI Engineer Notu: Ekran görüntülerini algılayan, akıl yürüten ve fare/klavye kullanarak API'ye ihtiyaç duymadan web siteleri ve masaüstü uygulamalarında insan gibi görev gerçekleştirebilen modern agent.

9. Voice Agent (Sesli Yapay Zekâ Ajanı)

Nedir? Kullanıcıyla sesli iletişim kurabilen, konuşmayı anlayan, yorumlayan, gerektiğinde Tool kullanan ve doğal sesli cevap verebilen agent.

Neden ortaya çıktı? Eskiden sesli sistemler "1'i tuşlayın, 2'yi tuşlayın" gibi önceden belirlenmiş senaryoları takip ediyordu. Voice Agent gerçek bir konuşma yapabilir.

Akış:

Kullanıcı Konuşur → Speech-to-Text (STT) → LLM → Reasoning → Tool Calling (gerekirse) → Text-to-Speech (TTS) → Sesli Cevap

Temel bileşenleri:

1. Speech Recognition (STT) — sesi metne çevirir ("Yarın saat 3'e toplantı oluştur")
2. Natural Language Understanding — kelimeleri değil niyeti anlar ("Yarın saat 3 boş muyum?" → takvime bakması gerektiğini anlar)
3. Reasoning — takvime mi bakmalı, mail mi göndermeli, Tool mu kullanmalı karar verir
4. Tool Calling — Calendar, CRM, SQL, Weather, MCP Server gibi araçları kullanır
5. Text-to-Speech (TTS) — cevabı tekrar sese dönüştürür, doğal konuşma duyulur

Temel özellikler: doğal konuşma, context awareness (örn. "Ankara'nın hava durumu nasıl?" sonra "yarın?" — "yarın"ın Ankara için sorulduğunu anlar), multi-turn conversation, real-time response, tool integration (takvim/mail/CRM/RAG/SQL/web search).

Örnekler:

- Çağrı merkezi: müşteri → Voice Agent → CRM → sipariş bilgisi → sesli cevap
- Banka: "hesap bakiyemi söyler misin?" → kimlik doğrulama → banka API → sesli cevap
- IT destek: "VPN'e bağlanamıyorum" → sorunu analiz et → bilgi tabanını tara → çözümü sesli anlat
- Restoran: "bu akşam 8'e rezervasyon yap" → rezervasyon sistemi → onay → sesli cevap

Avantajları: doğal iletişim sağlar, eller serbest kullanım sunar, 7/24 hizmet verebilir, aynı anda binlerce görüşmeyi yönetebilir, bekleme sürelerini azaltır.

Dezavantajları: gürültülü ortamda hata yapabilir, farklı aksanları anlamakta zorlanabilir, yanlış anlama riski var, gizlilik/ses verisi güvenliği önemli.

AI Engineer Notu: Konuşmayı metne dönüştüren (STT), niyeti anlayan, gerektiğinde Tool Calling yapan ve cevabı tekrar sese (TTS) dönüştüren, insanlarla gerçek zamanlı sesli iletişim kurabilen modern agent.

10. Embodied Agent (Embodied AI)

Nedir? Yalnızca dijital ortamda değil; gerçek veya simüle edilmiş fiziksel bir ortamı algılayabilen, bu ortamda hareket edebilen ve fiziksel görevleri yerine getirebilen agent. Fiziksel/sanal bir bedene sahip olup çevresini algılayan, karar veren ve fiziksel dünyada hareket ederek görev yapan agent.

Neden ortaya çıktı? Şimdiye kadarki agent'lar (kod yazan, internette gezen, bilgisayar kullanan, sesli konuşan) hepsi dijital dünyada çalışıyordu. Bir robotun yürümesi, kapı açması, kutu kaldırması, masayı toplaması gerekirse — bunun için Embodied Agent geliştirildi.

Temel fikir: bir bedeni vardır — gerçek robot, insansı robot, drone, otonom araç veya simülasyondaki sanal robot olabilir. Agent artık sadece düşünüp cevap vermez, fiziksel hareket de eder.

Akış:

Kamera/Lidar/Mikrofon/Sensörler → Perception → Reasoning → Planning → Motor Komutları → Robot Hareketi → Yeni Sensör Verisi → Tekrar Karar Ver
Temel bileşenleri:

1. Perception — kamera, lidar, derinlik kamerası, mikrofon, dokunma sensörleri, GPS, IMU ile çevreyi algılar
2. Computer Vision — kamera görüntüsünü analiz eder (insanı, kapıyı, masayı, bardağı tanır); Object Detection, Pose Estimation, Segmentation kullanılır
3. Reasoning — "kırmızı bardağı al" → bardağı bul → ulaşılabilir mi? → robot kolunu nasıl hareket ettireyim?
4. Planning — görevi adımlara böler (yürü → masaya yaklaş → kolu uzat → bardağı kavra → kaldır)
5. Action — motorlara komut gönderir, robot gerçekten hareket eder

Örnekler: Ev robotu: "mutfağı temizle" → mutfağı tara → çöpleri bul → topla → çöp kutusuna götür

- Otonom araç: kamera → yaya algıla → fren kararı → aracı durdur
- Depo robotu: kutuyu bul → konumu hesapla → robot kolunu hareket ettir → rafa yerleştir
- Tarım robotu: bitkileri tara → hastalıklı bitkiyi bul → ilaçla

Kullandığı teknolojiler: Computer Vision, Object Detection, Pose Estimation, SLAM, Reinforcement Learning, Planning, LLM, Tool Calling, Robot Control.

Embodied AI vs Robotics: Robotics robotun donanımıyla ilgilenir (motor, mekanik, elektronik, sensör) — vücut. Embodied AI robotun beynidir — algılar, düşünür, karar verir, hareketi planlar.

Avantajları: fiziksel görevleri yerine getirebilir, gerçek zamanlı karar verebilir, sürekli çevreden öğrenebilir, insan müdahalesini azaltır.

Dezavantajları: donanım maliyeti yüksek, güvenlik kritik öneme sahip, gerçek dünya dijital ortama göre çok daha karmaşık, hatalı kararlar fiziksel zarara yol açabilir.

Gerçek hayattaki örnekler: Tesla Optimus, Boston Dynamics Atlas, Figure AI, Waymo, Agility Robotics, NVIDIA Isaac Platform.

AI Engineer Notu: Kamera ve diğer sensörlerle çevresini algılayan, akıl yürüten, hareket planı oluşturan ve robotik bir gövde veya sanal avatar üzerinden fiziksel dünyada görev gerçekleştirebilen agent türü.

Modern LLM Agent Türleri Karşılaştırma Tablosu

Agent Türü	Temel Görevi	Çalıştığı Ortam	En Belirgin Yeteneği	Örnek
Tool-Using Agent	Dış araçları (API, SQL, MCP, Calendar vb.) kullanır.	Dijital	Tool Calling	Hava durumu sorgulama, takvim oluşturma
ReAct Agent	Düşün → Eylem → Gözlem döngüsüyle problem çözer.	Dijital	Adım adım akıl yürütme	Web araştırması yaparken her sonuca göre yeni karar vermesi
Planning Agent	Büyük görevleri küçük adımlara ayırır ve plan oluşturur.	Dijital	Task Planning	"Seyahat planı hazırla" görevini alt adımlara bölmesi
Autonomous Agent	İnsan müdahalesi olmadan hedefe ulaşmaya çalışır.	Dijital	Otonom çalışma	Bir görevi baştan sona kendi tamamlaması
Retrieval Agent	Doğru bilgi kaynağını bulur ve LLM'e bağlam sağlar.	RAG / Veri Kaynakları	Bilgi toplama	PDF, SQL, Web veya API'den bilgi getirerek cevap üretmesi
Coding Agent	Yazılım geliştirir, kod düzenler ve test çalıştırır.	Yazılım Geliştirme	Kod üretme ve doğrulama	Bug düzeltme, test çalıştırma, yeni özellik geliştirme
Browser Agent	Web sitelerinde gezinir ve internet görevlerini yapar.	Web Tarayıcısı	Web Automation	Fiyat karşılaştırma, rezervasyon araştırma
Computer Use Agent	Bilgisayarı insan gibi kullanır.	Tüm İşletim Sistemi	Fare ve klavye kontrolü	Excel kullanma, VS Code açma, Gmail gönderme
Voice Agent	İnsanlarla doğal sesli iletişim kurar.	Sesli Sistemler	STT + LLM + TTS	Çağrı merkezi, sesli asistan
Embodied Agent	Fiziksel dünyayı algılar ve hareket eder.	Robot / Simülasyon	Robotik hareket	Robot kolu, otonom araç, insansı robot

3.Multi-Agent Mimarileri

1. Multi-Agent System
2. Hierarchical Agent
3. Collaborative Agent
4. Supervisor Pattern
5. Manager–Worker
6. Handoffs
7. Agents as Tools

1. Multi-Agent System (MAS)

Nedir?

Multi-Agent System (MAS), tek bir agent yerine birden fazla uzman AI Agent'ın ortak bir hedef için birlikte çalıştığı mimaridir. Her agent kendi uzmanlık alanındaki görevi yerine getirir ve elde edilen sonuçlar birleştirilerek nihai çıktı oluşturulur. IBM'e göre bu yapı, tek bir agent'ın yönetmekte zorlanacağı karmaşık görevleri daha verimli çözmek için geliştirilmiştir.

Kısaca: "Tek agent her işi yapar." yerine "Her işi o konuda uzman agent yapar."

Neden Ortaya Çıktı?

İlk agent sistemlerinde tek bir AI Agent;

- plan yapıyor,
- araştırma yapıyor,
- tool kullanıyor,
- kod yazıyor,
- rapor hazırlıyordu.

Görevler büyüdükçe bu yaklaşım yetersiz kalmaya başladı. Çünkü tek agent'ın hem bağlamı büyüyor hem de her konuda uzman olması bekleniyordu.

Bu nedenle görevler küçük parçalara ayrılarak farklı uzman agent'lara dağıtılmaya başlandı. Böylece sistem hem daha ölçeklenebilir hem de daha doğru çalışır hale geldi.

Nasıl Çalışır?

Bir Multi-Agent sisteminde önce kullanıcıdan gelen görev analiz edilir. Ardından görev alt parçalara bölünür ve her parça ilgili uzman agent'a gönderilir. Agent'lar kendi görevlerini tamamladıktan sonra sonuçlar tekrar birleştirilerek kullanıcıya tek bir cevap olarak sunulur.

Kullanıcı
↓
Görev Analizi
↓
Alt Görevlere Böl
↓
Research Agent · Planning Agent · Coding Agent · Browser Agent
↓
Sonuçları Birleştir
↓
Final Cevap

Gerçek Hayattan Örnek

Kullanıcı: "İstanbul için 3 günlük AI konferansı seyahati planla."

Sistem bunu tek agent'a yaptırmak yerine;

- **Planning Agent** → Planı oluşturur.
- **Flight Agent** → Uçuşları araştırır.
- **Hotel Agent** → Otelleri bulur.
- **Budget Agent** → Toplam maliyeti hesaplar.

Son olarak tüm bilgiler birleştirilerek kullanıcıya tek bir seyahat planı sunulur.

Avantajları

- Her agent belirli bir konuda uzmanlaşır.
- Paralel çalışabildiği için daha hızlıdır.
- Büyük ve karmaşık görevleri daha rahat yönetebilir.
- Sisteme yeni agent eklemek kolaydır.
- Modüler yapısı sayesinde bakım ve geliştirme daha kolaydır.

Dezavantajları

- Agent'lar arasında koordinasyon gerekir.
- İletişim maliyeti oluşur.
- Mimari tek agent'a göre daha karmaşıktır.
- Yanlış koordinasyon tüm sistemi etkileyebilir.

AI Engineer Notu

Multi-Agent System (MAS), karmaşık problemleri çözmek amacıyla farklı görevlerde uzmanlaşmış birden fazla AI Agent'ın koordineli biçimde birlikte çalıştığı mimaridir. Bu yapı sayesinde görevler bölünür, paralel yürütülür ve tek agent'a göre daha ölçeklenebilir çözümler elde edilir.

2. Hierarchical Agent

Nedir?

Hierarchical Agent, agent'ların belirli bir hiyerarşi içerisinde organize edildiği Multi-Agent mimarisidir. Bu yapıda üst seviyedeki agent'lar stratejik kararları alırken, alt seviyedeki agent'lar kendilerine verilen görevleri yerine getirir. Böylece büyük ve karmaşık problemler daha küçük görevlere bölünerek yönetilebilir.

Kısaca: Patron–yönetici–çalışan mantığıyla çalışan Multi-Agent mimarisidir.

Neden Kullanılır?

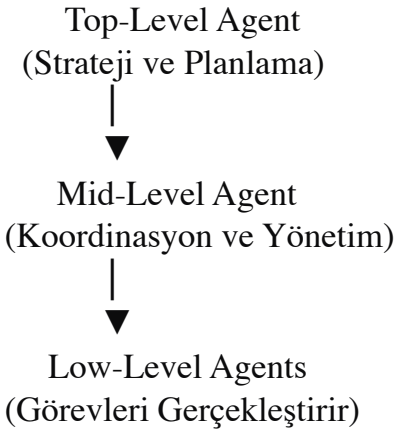
Tek bir agent'ın bütün süreci yönetmesi yerine görevler farklı seviyelere ayrılır.

- **Üst seviye** → Plan yapar.
- **Orta seviye** → Planı organize eder.
- **Alt seviye** → Görevleri uygular.

Bu yapı özellikle büyük ölçekli sistemlerde koordinasyonu kolaylaştırır ve agent'ların kendi uzmanlık alanlarına odaklanmasını sağlar.

Yapısı

IBM'e göre hiyerarşik mimariler genellikle iki veya üç katmandan oluşur.



Bazı sistemlerde orta katman bulunmaz ve üst seviye agent doğrudan alt seviye agent'ları yönetir.

Agent Rolleri

Top-Level Agent — En üst seviyedeki agent'tır.

- Kullanıcı isteğini yorumlar.
- Genel stratejiyi belirler.
- Görevi alt parçalara ayırır.
- Hangi agent'ın hangi işi yapacağına karar verir.

Büyük resmi gören agent budur.

Mid-Level Agent — Üst seviyeden gelen planı operasyonel görevlere dönüştürür.

- Alt agent'ları yönetmek,
- Görev dağıtımı yapmak,
- Sonuçları toplamak,
- İlerlemeyi takip etmek.

Bir nevi takım lideridir.

Low-Level Agent — Sistemin uygulayıcı agent'larıdır. Her biri belirli bir konuda uzmanlaşmıştır (Browser Agent, Coding Agent, Retrieval Agent, Database Agent gibi). Görevlerini tamamladıktan sonra sonuçlarını üst seviyeye rapor ederler.

Nasıl Çalışır?

Kullanıcı
↓
Top-Level Agent (Görevi Analiz Et)
↓
Görevi Alt Parçalara Böl
↓
Mid-Level Agent (Görevleri Dağıt)
↓
Coding Agent · Browser Agent · Retrieval Agent
↓
Sonuçları Topla
↓
Final Cevap

Gerçek Hayattan Örnek

Kullanıcı: "Şirketin geçen çeyrek satış performansını analiz edip rapor hazırla."

- **Top-Level Agent** → Analizin nasıl yapılacağını planlar.
- **Mid-Level Agent** → SQL Agent, Analysis Agent ve Report Agent'a görev dağıtır.
- **SQL Agent** → Verileri çeker.
- **Analysis Agent** → Analizleri yapar.
- **Report Agent** → Son raporu oluşturur.

Sonuç tekrar üst seviyeye gönderilir ve kullanıcıya sunulur.

Avantajları

- Büyük görevleri kolay yönetir.
- Net görev dağılımı sağlar.
- Uzmanlaşmayı artırır.
- Modüler olduğu için yeni agent eklemek kolaydır.
- Paralel çalışma sayesinde verimlilik sağlar.
- Bir alt agent başarısız olursa görev başka bir agent'a yeniden atanabilir.

Dezavantajları

- Mimari tasarımı daha karmaşıktır.
- Üst seviyedeki agent darboğaz oluşturabilir.
- Katı hiyerarşi çok dinamik ortamlarda yavaş kalabilir.
- Agent'lar arasındaki iletişim gecikmeleri tüm sistemi etkileyebilir.

Kullanım Alanları

Başlıca kullanım alanları: tedarik zinciri ve lojistik, akıllı üretim sistemleri, Agentic RAG, siber güvenlik, otonom araç filoları, teslimat robotları.

AI Engineer Notu

Hierarchical Agent, görevleri stratejik, operasyonel ve uygulama katmanlarına ayırarak çalışan çok katmanlı bir Multi-Agent mimarisidir. Üst seviye agent planlama yapar, orta seviye koordinasyonu sağlar, alt seviye agent'lar ise uzmanlık alanlarındaki görevleri yerine getirir.

3. Collaborative Agent

Nedir?

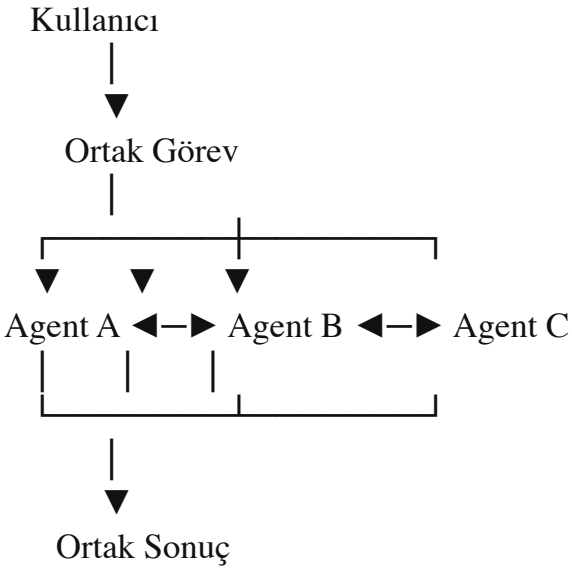
Collaborative Agent, birden fazla AI Agent'ın eşit seviyede çalışarak, ortak bir hedefe ulaşmak için bilgi paylaşması, birlikte karar vermesi ve görevleri koordineli şekilde yürütmesini sağlayan Multi-Agent mimarisidir. Bu yapıda belirgin bir yönetici veya lider agent bulunmaz; agent'lar birbirleriyle iletişim kurarak ortak çözüme ulaşırlar. **Kısaca:** Lider yoktur; tüm agent'lar takım arkadaşları gibi birlikte çalışır.

Neden Kullanılır?

Bazı problemlerde tek bir merkezi yönetici kullanmak verimsiz olabilir. Özellikle büyük, dağıtık ve sürekli değişen sistemlerde her agent bulunduğu ortam hakkında yerel bilgiye sahiptir. Bu nedenle agent'lar kararlarını tek başına değil, birbirleriyle iletişim kurarak verirler.Bu yaklaşım sistemin daha esnek, ölçeklenebilir ve hata toleranslı olmasını sağlar.

Nasıl Çalışır?

Collaborative Agent mimarisinde sistem önce görevi analiz eder ve uygun agent'lara dağıtır. Agent'lar görevlerini yerine getirirken birbirleriyle sürekli bilgi paylaşır, gerektiğinde sorumluluk değiştirir ve ortak bir sonuca ulaşırlar.



Buradaki en önemli nokta, agent'ların birbirleriyle doğrudan iletişim kurabilmesidir.

Temel Özellikleri

Eşit Yetki — Hiçbir agent diğerinden üstün değildir. Her agent kendi uzmanlığı doğrultusunda karar verebilir.

Sürekli İletişim — Agent'lar durum bilgisi paylaşır, görev paylaşımı yapar, birbirlerinin sonuçlarını kullanabilir, gerektiğinde planlarını günceller. Bu sayede sistem tek bir merkezden yönetilmese bile uyumlu şekilde çalışabilir.

Ortak Karar Verme — Her agent yalnızca kendi görevini yapmakla kalmaz, diğer agent'lardan gelen bilgiler doğrultusunda kendi kararlarını da güncelleyebilir. Bu nedenle sistem sürekli gelişen ortak bir çözüm üretir.

Gerçek Hayattan Örnek

Bir afet bölgesinde çalışan drone filosunu düşünelim. Her drone; farklı bir bölgeyi tarar, bulduğu bilgileri diğer dronelarla paylaşır, çakışmayı önlemek için rotasını değiştirir, yeni görevler üstlenebilir. Hiçbir drone diğerlerinin yöneticisi değildir. Ancak birlikte çalışarak çok daha geniş bir alanı kısa sürede tarayabilirler. Bu, Collaborative Agent mimarisinin en güzel örneklerinden biridir.

Avantajları

- Merkezi bir lidere bağımlı değildir.
- Esnek ve dinamik ortamlara kolay uyum sağlar.
- Agent'lar paralel çalışabilir.
- Tek bir agent'ın başarısız olması tüm sistemi durdurmaz.
- Büyük ve dağıtık sistemlerde yüksek ölçeklenebilirlik sunar.

Dezavantajları

- Agent'lar arasında yoğun iletişim gerekir.
- Koordinasyon daha zor olabilir.
- Çatışan kararların çözülmesi gerekebilir.
- Çok fazla agent olduğunda iletişim maliyeti artabilir.

Kullanım Alanları

Drone filoları, akıllı şehir sistemleri, otonom araçlar, robot ekipleri, dağıtık sağlık sistemleri, finansal analiz platformları, büyük ölçekli araştırma sistemleri.

AI Engineer Notu

Collaborative Agent, birden fazla AI Agent'ın lider olmadan, eşit seviyede bilgi paylaşarak ve birlikte karar vererek ortak bir hedefe ulaşmasını sağlayan Multi-Agent mimarisidir. Bu yapı özellikle dağıtık, dinamik ve gerçek zamanlı koordinasyon gerektiren sistemlerde tercih edilir.

4. Supervisor Pattern

Nedir?

Supervisor Pattern, bir Supervisor Agent'ın (yönetici agent) kullanıcıdan gelen isteği analiz ettiği, görevi uygun uzman Worker Agent'lara dağıttığı ve onların sonuçlarını birleştirerek kullanıcıya son cevabı sunduğu Multi-Agent mimarisidir. Bu yapıda Worker Agent'lar birbirleriyle iletişim kurmaz. Tüm iletişim ve koordinasyon Supervisor üzerinden gerçekleşir.

Kısaca: Tek bir yönetici agent görevleri uzman agent'lara dağıtır, sonuçları toplar ve final cevabı oluşturur.

Neden Kullanılır?

Karmaşık görevlerde hangi agent'ın hangi işi yapacağını her agent'ın kendisinin belirlemesi yerine, bu karar tek bir yönetici tarafından verilir. Bu sayede; görev dağılımı merkezi olarak yönetilir, sorumluluklar netleşir, sistem daha kolay izlenebilir, hangi agent'ın ne yaptığı kayıt altına alınabilir. Bu nedenle Supervisor Pattern günümüzde üretim ortamlarında en yaygın kullanılan Multi-Agent mimarilerinden biridir.

Nasıl Çalışır?

Supervisor önce kullanıcı isteğini analiz eder. Gerekirse görevi alt görevlere böler. Ardından uygun Worker Agent'ları seçer. Worker Agent'lar görevlerini tamamladıktan sonra sonuçlarını tekrar Supervisor'a gönderir. Supervisor bu sonuçları değerlendirir ve gerekirse yeni görevler oluşturabilir veya doğrudan final cevabı kullanıcıya iletir.

Kullanıcı
↓
Supervisor Agent
↓
Görevi Analiz Et
↓
Research Agent · Writer Agent · Coding Agent · Fact Checker
↓
Sonuçları Supervisor'a Gönder
↓
Supervisor Sonuçları Birleştir
↓
Final Cevap

En Önemli Özelliği

Worker Agent'lar birbirleriyle konuşmaz:

Research Agent ✕ Writer Agent ← doğrudan iletişim yok

Bunun yerine:

Research Agent → Supervisor → Writer Agent

şeklinde bütün iletişim Supervisor üzerinden yürütülür. Bu yapı koordinasyonu oldukça kolaylaştırır.

Gerçek Hayattan Örnek

Kullanıcı: "Yapay zekâ hakkında teknik bir makale hazırla."

Supervisor Agent şu görevleri dağıtır:

- **Research Agent** → Kaynakları araştırır.
- **Writer Agent** → Makaleyi yazar.
- **Fact Checker Agent** → Bilgileri doğrular.
- **Formatter Agent** → Son düzenlemeleri yapar.

Tüm sonuçlar tekrar Supervisor'a gelir ve Supervisor tek bir makale olarak kullanıcıya sunar.

Avantajları

- Merkezi koordinasyon sağlar.
- Görev dağılımı daha kontrollüdür.
- Uzman agent'ların yönetimini kolaylaştırır.
- Hangi agent'ın hangi işi yaptığı kolayca takip edilir.
- Denetlenebilir (audit) sistemler için oldukça uygundur.

Dezavantajları

- Supervisor tüm yükü taşıdığı için darboğaz oluşabilir.
- Çok fazla Worker Agent olduğunda doğru agent'ı seçmek zorlaşabilir.
- Supervisor başarısız olursa tüm sistem etkilenebilir.
- Bu nedenle büyük sistemlerde birden fazla Supervisor kullanılarak hiyerarşik yapılar oluşturulabilir.

Kullanım Alanları

Agentic RAG sistemleri, yazılım geliştirme agent'ları, kurumsal AI asistanları, belge analizi, araştırma ve raporlama sistemleri.

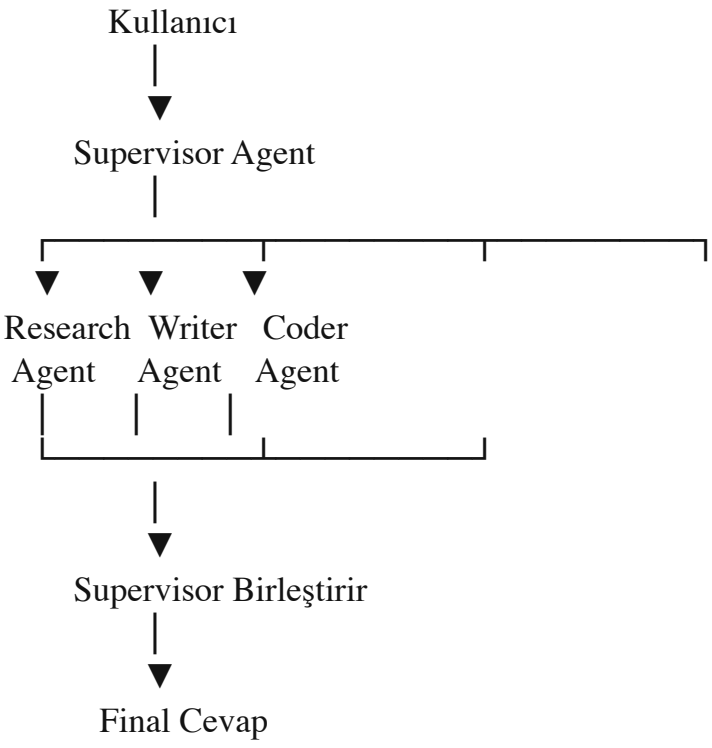
Hierarchical Agent ile Farkı

Bu iki kavram çok benzer görünür ancak aynı değildir. Supervisor Pattern, aslında Hierarchical Agent mimarisinin en yaygın kullanılan uygulamalarından biridir (genelde 2 katmanlı: Supervisor + Worker'lar).

AI Engineer Notu

Supervisor Pattern, kullanıcı isteğini analiz eden, görevleri uygun uzman Worker Agent'lara dağıtan ve onların sonuçlarını tek bir noktada birleştirerek nihai cevabı oluşturan merkezi koordinasyon tabanlı Multi-Agent tasarım desenidir. Worker Agent'lar birbirleriyle değil, yalnızca Supervisor ile iletişim kurar.

Akış



5. Manager–Worker Pattern

Nedir?

Manager–Worker Pattern, bir Manager Agent'ın büyük bir görevi küçük ve bağımsız parçalara bölerek bunları birden fazla Worker Agent'a dağıttığı ve gelen sonuçları birleştirerek nihai çıktıyı oluşturduğu Multi-Agent tasarım desenidir. Supervisor Pattern'a oldukça benzer görünse de burada odak noktası iş yükünü paralel olarak dağıtmak ve tüm Worker'ları mümkün olduğunca verimli kullanmaktır.

Kısaca: Manager işi böler, Worker'lar paralel çalışır, Manager sonuçları toplar.

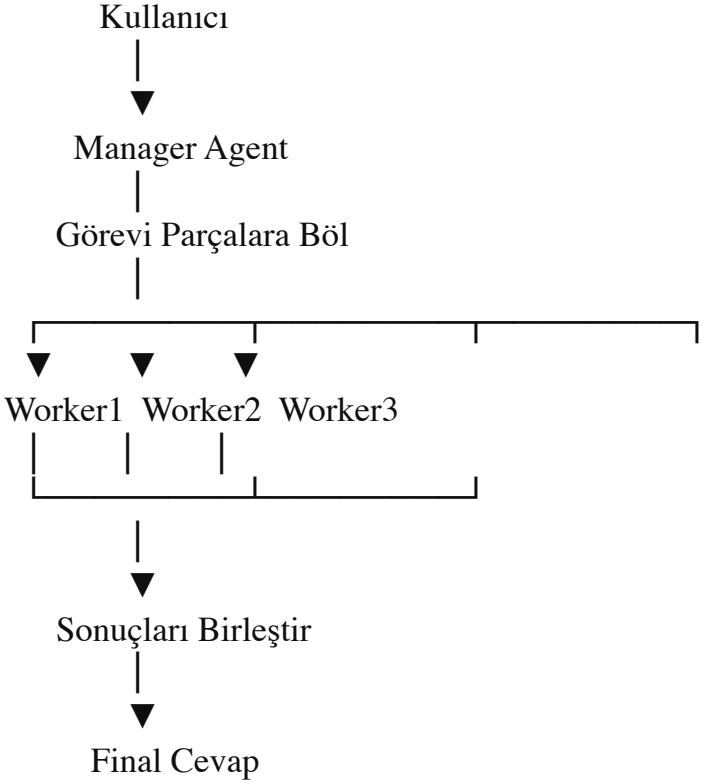
Neden Kullanılır?

Bazı görevler tek bir agent tarafından yapılabilecek kadar küçük değildir. Örneğin; milyonlarca dosyanın analiz edilmesi, binlerce PDF'in taranması, yüzlerce web sayfasının araştırılması, çok büyük veri kümelerinin işlenmesi gibi durumlarda işi tek agent'a vermek hem yavaş hem de verimsiz olur.

Manager–Worker mimarisinde büyük görev küçük parçalara ayrılır ve aynı anda birçok Worker tarafından işlenir. Böylece işlem süresi ciddi şekilde azalır.

Nasıl Çalışır?

Önce Manager kullanıcı isteğini alır. Görevi bağımsız alt görevlere böler. Her görevi uygun Worker Agent'a gönderir. Worker Agent'lar birbirinden bağımsız çalışır. Sonuçlar tekrar Manager'a gelir. Manager bütün sonuçları birleştirerek kullanıcıya final cevabı verir.



Push ve Pull Modelleri

Kaynağın özellikle üzerinde durduğu konu budur. Manager–Worker iki farklı şekilde çalışabilir.

1. Push Model — Bu modelde Manager aktiftir. Boşta duran Worker'ları bilir ve görevleri kendisi gönderir.

Manager → Worker 1 → Worker 2 → Worker 3

Yani işi dağıtan taraf Manager'dır. Bu yöntem Kubernetes, HPC kümeleri, kurumsal AI sistemleri gibi kontrolün merkezde olduğu yapılarda sık kullanılır.

2. Pull Model — Bu modelde ise Worker aktiftir. Worker "Ben boşum, bana yeni görev ver." diyerek Manager'dan iş ister.

Worker → Manager → Yeni Görev

Bu yöntem özellikle dağıtık sistemlerde, gönüllü bilgisayar ağlarında, dinamik Worker sayısının değiştiği ortamlarda daha verimlidir.

En Önemli Özelliği

Manager sadece görev dağıtmaz. Aynı zamanda; hangi görevin tamamlandığını, hangi Worker'ın çalıştığını, hangi görevin başarısız olduğunu, hangi görevin yeniden gönderileceğini takip eder. Yani sistemin koordinasyonundan tamamen Manager sorumludur.

Gerçek Hayattan Örnek

Kullanıcı: "10.000 PDF içindeki tüm sözleşmeleri analiz et."

Manager şöyle davranır:

- PDF 1–1000 → Worker 1
- PDF 1001–2000 → Worker 2
- PDF 2001–3000 → Worker 3
- ...

Her Worker kendi PDF grubunu analiz eder. Sonuçlar tekrar Manager'a gelir. Manager bunları tek rapor halinde kullanıcıya sunar.

Avantajları

- Büyük işleri paralel çalıştırır.
- Sistemi hızlandırır.
- Worker sayısı kolayca artırılabilir.
- Kaynak kullanımı daha verimlidir.
- Büyük veri işlemede oldukça başarılıdır.

Dezavantajları

- Manager başarısız olursa sistem durabilir.
- Worker'lar arasında iş yükü dengesiz olabilir.
- Çok fazla Worker olduğunda koordinasyon zorlaşabilir.
- Manager'ın görev takibini doğru yapması gerekir.

Kullanım Alanları

Büyük ölçekli veri analizi, AI Agent sistemleri, Agentic RAG, dağıtık hesaplama (Distributed Computing), MapReduce, HPC (High Performance Computing), Kubernetes tabanlı iş yükleri.

Supervisor Pattern ile Farkı

Bu iki desen birbirine çok benzer ancak amaçları farklıdır:

- **Supervisor Pattern:** Research Agent → Writer Agent → Fact Checker gibi farklı **uzman** agent'ları yönetir.
- **Manager–Worker Pattern:** Aynı analiz işini 100 Worker'a bölerek çok daha **hızlı** tamamlar.

AI Engineer Notu

Manager–Worker Pattern, büyük ve bağımsız görevleri küçük parçalara ayırarak bunları paralel çalışan Worker Agent'lara dağıtan ve sonuçları tekrar birleştiren Multi-Agent tasarım desenidir. Temel amacı uzmanlık yönetmekten çok, iş yükünü dağıtarak sistemi daha hızlı ve ölçeklenebilir hale getirmektir.

6. Handoffs

Nedir?

Handoffs, bir AI Agent'ın mevcut görevi kendisinden daha uygun veya daha uzman başka bir AI Agent'a devretmesini sağlayan mekanizmadır. Böylece her agent yalnızca uzman olduğu işi yapar ve görev doğru agent tarafından devam ettirilir.

OpenAI Agents SDK'de handoff'lar bir **Tool Calling** mekanizması olarak çalışır. Model, uygun gördüğü anda `transfer_to_refund_agent` gibi bir tool'u çağırır ve konuşmanın kontrolü ilgili agent'a geçer.

Kısaca: Bir agent, "Bu iş benim uzmanlık alanım değil." diyerek görevi doğru uzmana devreder.

Neden Kullanılır?

İlk bakışta şöyle düşünülebilir: "Zaten hepsi aynı LLM'i kullanıyorsa neden agent değiştiriyoruz?"

Aslında değişen LLM değil, agent'ın sahip olduğu özelliklerdir. Her agent'ın; farklı System Prompt'u, farklı Tool'ları, farklı yetkileri, farklı Memory'si, hatta gerekirse farklı LLM modeli olabilir.

Örneğin;

- **Refund Agent** → `refund_order()`, `check_payment()` gibi araçları kullanabilir.
- **Coding Agent** → GitHub, Terminal, Docker gibi araçlara erişebilir.
- **Browser Agent** → Web'de araştırma yapabilir.

Bu nedenle her agent yalnızca kendi uzman olduğu işi yapar. Böylece sistem hem daha doğru hem de daha verimli çalışır.

Nasıl Çalışır?

Bir müşteri destek sistemi düşünelim. **Kullanıcı:** "Siparişimi iade etmek istiyorum."

Kullanıcı
↓
Triage Agent (İlk karşılayan agent)
↓
"Bu bir iade işlemi."
↓
Handoff (Tool Call)
↓
Refund Agent
↓
Siparişi kontrol et → Ödemeyi doğrula → İade işlemini başlat
↓
Final Cevap

Dikkat edilmesi gereken nokta: **Triage Agent iade işlemini yapmaz.** Sadece kullanıcının ne istediğini anlar ve görevi ilgili uzmana devreder. İşlemi ise Refund Agent gerçekleştirir.

OpenAI Agents SDK'de Handoff

OpenAI Agents SDK'de her Agent'ın bir `handoffs` listesi bulunur.

python
billing_agent = Agent(name="Billing Agent")
refund_agent = Agent(name="Refund Agent")

triage_agent = Agent(

```
    name="Triage Agent",
    handoffs=[billing_agent, refund_agent]
)
```
Kullanıcı mesajı geldiğinde model; Billing Agent'a mı, Refund Agent'a mı geçmesi gerektiğine kendisi karar verir.

Handoff Bir Tool Gibi Çalışır

OpenAI Agents SDK'de handoff aslında özel bir Tool Calling işlemidir. Örneğin `transfer_to_refund_agent()` isimli bir tool çağrılır. Bu tool çalıştığında konuşmanın kontrolü Refund Agent'a geçer.

Tool Calling → Agent Değiştir → Yeni Agent konuşmaya devam eder.

Handoff Sırasında Veri Gönderme

Handoff yapılırken yeni agent'a ek bilgiler de gönderilebilir. Örneğin: `Reason → duplicate_charge`, `Priority → high`. Bu bilgiler sayesinde yeni agent göreve başlamadan önce gerekli bağlamı edinmiş olur.

Input Filter

Varsayılan olarak yeni agent önceki konuşma geçmişini görür. Ancak Input Filter kullanılarak;

- Tool çağrıları kaldırılabilir.
- Gereksiz mesajlar temizlenebilir.
- Konuşmanın yalnızca özeti gönderilebilir.

Bu yöntem özellikle büyük konuşmalarda token maliyetini azaltmak için kullanılır.

Gerçek Hayattan Örnek

Bir banka chatbot'u düşünelim. **Kullanıcı:** "Kartımdan iki kez para çekilmiş."

Customer Agent
↓
Sorunu Analiz Et
↓
Handoff
↓
Refund Agent
↓
İade Sürecini Başlat
↓
Final Cevap

Customer Agent problemi anlar. Refund Agent ise işlemi gerçekleştirir.

Avantajları

- Her agent yalnızca uzman olduğu işi yapar.
- Daha modüler sistemler oluşturulur.
- Yeni agent eklemek kolaydır.
- Sistem daha doğru kararlar verir.
- Tool Calling mantığıyla doğal şekilde çalışır.

Dezavantajları

- Çok fazla handoff yapılırsa gecikme oluşabilir.
- Yanlış agent seçimi performansı düşürebilir.
- Konuşma geçmişinin doğru yönetilmesi gerekir.
- Fazla agent olduğunda geçişlerin yönetimi zorlaşabilir.

AI Engineer Notu

Handoffs, bir AI Agent'ın kullanıcı isteğini analiz ederek görevi daha uygun bir uzman AI Agent'a devretmesini sağlayan mekanizmadır. OpenAI Agents SDK'de bu işlem Tool Calling mantığıyla gerçekleştirilir ve konuşmanın kontrolü ilgili agent'a aktarılır.

Akış

Kullanıcı → Triage Agent → Görevi Analiz Et → Handoff (Tool Call)
→ Uzman Agent → Görevi Tamamla → Final Cevap

Çok Önemli Bir Nokta

Bu dokümandaki en önemli bilgi şu: **OpenAI Agents SDK'de Handoff aslında özel bir Tool Calling mekanizmasıdır.** LLM, `transfer_to_refund_agent` gibi bir tool'u çağırır ve böylece konuşmanın kontrolü yeni agent'a geçer. Bu yüzden Handoff, teorik olarak "agent değiştirme", teknik olarak ise bir tool çağrısı şeklinde uygulanır. **Bu detay mülakatlarda da sık sorulan noktalardan biridir.**

7. Agents as Tools

Agents as Tools, bir AI Agent'ın başka bir AI Agent'ı normal bir Tool (fonksiyon) gibi çağırmasını sağlayan Multi-Agent tasarım desenidir. Bu yapıda dıştaki agent (**Outer Agent**) konuşmayı yönetmeye devam eder. İhtiyaç duyduğunda uzman bir agent'ı (**Inner Agent**) tool gibi çalıştırır, sonucunu alır ve konuşmaya kaldığı yerden devam eder.

Kısaca: Başka bir agent, Tool gibi kullanılır; konuşmanın kontrolü ise Outer Agent'da kalır.

Neden Kullanılır?

Bir agent büyüdükçe; prompt'u uzar, tool sayısı artar, uzmanlık alanı genişler, doğru tool'u seçmesi zorlaşır. Bu nedenle bütün işleri tek agent'a yüklemek yerine, belirli konular için

küçük uzman agent'lar oluşturulur. Outer Agent yalnızca gerektiğinde bu uzman agent'ları tool olarak çağırır. Böylece sistem daha modüler, okunabilir ve yönetilebilir hale gelir.

Nasıl Çalışır?

Kullanıcı: "Paris'e uçak bileti al ve masraf raporumu oluştur."

Outer Agent isteği analiz eder. Önce Travel Agent'ı tool olarak çağırır. Travel Agent kendi tool'larını kullanarak uçuşu ayarlar ve sonucu döndürür. Ardından Outer Agent bu sonucu alır ve Expense Agent'ı tool olarak çağırır. Expense Agent masraf kaydını oluşturur. Son olarak Outer Agent iki sonucu birleştirerek kullanıcıya tek cevap verir.

Kullanıcı
↓
Outer Agent
↓
Travel Agent (Tool) → Uçuşu Rezerve Et
↓
Outer Agent
↓
Expense Agent (Tool) → Masraf Kaydı Oluştur
↓
Outer Agent
↓
Final Cevap

En Önemli Mantığı

Inner Agent konuşmayı devralmaz. Sadece verilen görevi yapar ve sonucunu geri döndürür.
Konuşmaya devam eden her zaman Outer Agent'dır.

Handoffs ile Farkı

En çok karıştırılan konu budur.

Handoff:

Triage Agent → Refund Agent → Refund Agent konuşmaya devam eder.
Kontrol artık Refund Agent'dadır.

Agents as Tools:

Outer Agent → Travel Agent (Tool) → Sonucu Döndür → Outer Agent konuşmaya devam eder.
Kontrol hiç değişmez. Outer Agent sadece yardım almıştır.

Gerçek Hayattan Örnek

Bir kişisel asistan düşünelim. **Kullanıcı:** "Londra'ya uçak bileti al ve otel rezervasyonu yap."

Outer Agent önce Flight Agent'ı tool olarak çağırır. Flight Agent uçuşları bulur, rezervasyonu yapar, sonucu döndürür. Sonra Outer Agent Hotel Agent'ı tool olarak çağırır. Hotel Agent oteli ayarlar, sonucu döndürür. Outer Agent ise tüm sonuçları birleştirerek kullanıcıya tek cevap verir.

Avantajları

- Tek agent'ın aşırı büyümesini engeller.
- Uzman agent'lar oluşturmayı kolaylaştırır.
- Kod daha modüler olur.
- Her agent kendi tool ve prompt'una sahip olabilir.
- Outer Agent tüm süreci yönetmeye devam eder.

Dezavantajları

- Her Inner Agent ayrı bir LLM çağrısı yaptığı için gecikme oluşabilir.
- Yanlış Tool açıklamaları modelin yanlış agent'ı çağırmasına neden olabilir.
- Çok fazla iç içe agent çağrısı maliyeti artırabilir.

Dikkat Edilmesi Gerekenler

OpenAI dokümanına göre:

- Inner Agent kendi prompt'una, tool'larına ve LLM çağrısına sahiptir.
- Outer Agent, Inner Agent'ın nasıl çalıştığını görmez. Sadece nihai sonucu alır.
- Inner Agent, Outer Agent'ın tüm konuşma geçmişini otomatik olarak görmez; yalnızca kendisine gönderilen bilgileri alır.

AI Engineer Notu

Agents as Tools, bir AI Agent'ın başka bir AI Agent'ı normal bir fonksiyon (tool) gibi çağırmasını sağlayan Multi-Agent tasarım desenidir. Inner Agent yalnızca kendisine verilen görevi yerine getirir ve sonucu Outer Agent'a döndürür. **Konuşmanın kontrolü hiçbir zaman Outer Agent'dan ayrılmaz.**

Multi-Agent Mimarileri —Karşılaştırma Tablosu

Mimari	Lider var mı? / Kim Kontrol Eder?	Agent'lar Birbiriyle Konuşur mu?	Temel Amaç	En Çok Kullanıldığı Yer
Multi-Agent System (MAS)	Değişken — mimariye göre değişir	Değişken — mimariye göre değişir	Birden fazla uzman agent'ın ortak bir hedef için birlikte çalışmasını sağlamak	Genel Multi-Agent sistemleri
Hierarchical Agent	Evet (çok katmanlı) — genellikle Top-Level (üst seviye) Agent	Hayır — üst katman üzerinden dolaylı iletişim kurulur	Görevleri katmanlara ayırarak hiyerarşik şekilde yönetmek	Büyük kurumsal AI sistemleri, üretim, lojistik
Collaborative Agent	Hayır — merkezi lider yoktur	Evet (eşit seviyede)	Agent'ların eşit seviyede bilgi paylaşarak ortak karar vermesini sağlamak	Drone filoları, robot ekipleri, dağıtık sistemler
Supervisor Pattern	Evet (tek katman) — Supervisor Agent	Hayır — tüm iletişim Supervisor üzerinden gerçekleşir	Uzman Worker Agent'lara görev dağıtmak ve sonuçları tek noktada toplamak	LangGraph, OpenAI Agents SDK, Agentic RAG
Manager–Worker Pattern	Evet — Manager Agent	Genellikle Hayır	Büyük görevleri küçük parçalara ayırıp paralel olarak çalıştırmak (iş yükü dağıtımı)	Büyük veri işleme, HPC, dağıtık hesaplama, Kubernetes
Handoffs	Yok (kontrol devri) — devir sonrası görevi devralan Agent	Gerekmez	Görevi kendisinden daha uygun bir uzman agent'a tamamen devretmek	OpenAI Agents SDK, müşteri destek sistemleri
Agents as Tools	Outer Agent (kontrol her zaman onda kalır)	Hayır — Inner Agent yalnızca sonucu döndürür	Başka bir agent'ı sıradan bir Tool (fonksiyon) gibi çağırıp yardım almak	Modüler AI Agent sistemleri, OpenAI Agents SDK

Agent Workflow (Design Patterns), Frameworkler ve Ekosistem Kavramları

1. Workflow Nedir?
2. Prompt Chaining
3. Routing
4. Parallelization
5. Orchestrator–Workers
6. Evaluator–Optimizer
7. Agent Framework'ler
8. Agent Ekosistemindeki Güncel Kavramlar

Workflow Nedir?

Workflow (iş akışı), bir AI Agent'ın veya birden fazla AI Agent'ın bir görevi hangi sırayla, hangi yöntemlerle ve hangi karar mekanizmasıyla tamamlayacağını belirleyen çalışma düzenidir.

Başka bir ifadeyle: Workflow, agent sisteminin görev çözme yol haritasıdır.

Bir kullanıcı isteği geldiğinde sistemin;

- görevi nasıl analiz edeceği,
- hangi agent'ı veya modeli kullanacağı,
- işi hangi adımlara ayıracağı,
- hangi işlemleri aynı anda çalıştıracağı,
- hangi tool'ları çağıracağı,
- sonuçları nasıl birleştireceği,
- çıktıyı nasıl kontrol edip düzelteceği

workflow tarafından belirlenir. Bu nedenle **workflow doğrudan bir agent türü değildir** — agent'ın veya agent ekibinin görevi nasıl yürüteceğini belirleyen tasarım desenidir.

```
Kullanıcı İsteği
      ↓
İsteği Analiz Et
      ↓
Doğru Agent'ı Seç
      ↓
Görevi Adımlara Ayır
      ↓
Bağımsız İşleri Paralel Çalıştır
      ↓
Sonuçları Birleştir
      ↓
Çıktıyı Kontrol Et
      ↓
Final Cevap
```

Bu akışın her bölümünde farklı bir workflow pattern kullanılabilir. Agent workflow'ları genel olarak agent'ın planlama, tool çağırma, yürütme, gözlemleme ve gerektiğinde yaklaşımını değiştirme biçimini yapılandırır. İş akışındaki bir düğüm (node); LLM çağrısı, fonksiyon çağrısı, RAG işlemi veya başka bir görev olabilir.

Workflow'lar Neden Kullanılır?

Basit görevlerde tek bir LLM çağrısı yeterli olabilir. Örneğin: "*Bu cümleyi İngilizceye çevir.*" — burada doğrudan modelden cevap alınabilir.

Ama gerçek hayattaki görevler genellikle daha karmaşıktır. Örneğin: "*Şirketin satış raporlarını incele, güncel sektör haberlerini araştır, rakiplerle karşılaştır, riskleri belirle ve bana sunum hazırla*."

Bu görev tek bir prompt içerisinde yapılmaya çalışılırsa model;

- bazı adımları atlayabilir,
- bilgileri karıştırabilir,
- yanlış tool seçebilir,
- uzun bağlam nedeniyle odağını kaybedebilir,
- çıktının kalitesini kontrol edemeyebilir.

Workflow'lar büyük görevi yapılandırarak bu problemleri azaltır. Sistem görevi adımlara bölebilir, doğru uzmana yönlendirebilir, bağımsız görevleri eş zamanlı çalıştırabilir ve son çıktıyı değerlendirerek iyileştirebilir.

Bu bölümde incelenen beş temel workflow tasarım deseni: **Prompt Chaining, Routing, Parallelization, Orchestrator–Workers, Evaluator–Optimizer.**

1. Prompt Chaining

Prompt Chaining, büyük ve karmaşık bir görevi daha küçük, yönetilebilir adımlara bölerek her adımı sırayla tamamlayan workflow tasarım desenidir. **Her adımın çıktısı, bir sonraki adımın girdisi olur.** Yani tek bir prompt içinde her şeyi yaptırmak yerine, görev bir zincir şeklinde ilerletilir.

Kısaca: Büyük görevi küçük adımlara böl ve her adımı sırayla tamamla.

Neden Kullanılır?

LLM'e aynı anda çok sayıda görev verildiğinde modelin odağı dağılabilir. Örneğin: "*Bu PDF'i oku, özetle, önemli kavramları bul, başlıkları çıkar, sunuma dönüştür ve konuşma metni hazırla*."

Bunu tek prompt ile istemek mümkündür ancak model;

- bazı adımları atlayabilir,
- yeterince ayrıntılı çalışmayabilir,
- bilgileri yanlış sıraya koyabilir,
- çıktı formatını bozabilir.

Prompt Chaining ile model her aşamada yalnızca **tek bir göreve** odaklanır.

Nasıl Çalışır?

```
PDF
 ↓
Metni Çıkar
 ↓
```

```
Özet Oluştur
 ↓
Ana Başlıkları Belirle
 ↓
Slayt Yapısını Hazırla
 ↓
Slayt Metinlerini Yaz
 ↓
Final Sunum
```
Her adım bir önceki adımın çıktısını kullanır.

RAG Sistemindeki Benzeri

```
Kullanıcı Sorusu → Soru Embedding'i → Retriever → İlgili Chunk'lar
   → Prompt Oluşturma → LLM → Cevap
```
Her adım bir sonraki adıma veri sağladığı için yapısal olarak bir iş akışı zinciri oluşur.

Önemli ayrım: RAG akışındaki her adım mutlaka ayrı bir prompt değildir. Prompt Chaining ifadesi özellikle **birden fazla LLM/prompt adımının** birbirine bağlanmasını anlatır.

Gerçek Hayattan Örnek

Kullanıcı: "Bu müşteri yorumlarından kapsamlı bir analiz hazırla."

```
Yorumları Temizle
   ↓
Olumlu ve Olumsuz Olarak Sınıflandır
   ↓
En Sık Şikâyetleri Bul
   ↓
Genel Özet Oluştur
   ↓
Aksiyon Önerileri Yaz
```
Avantajları

- Büyük görevleri daha yönetilebilir hale getirir.
- LLM'in her adımda tek bir konuya odaklanmasını sağlar.
- Hata ayıklamayı kolaylaştırır.
- Her aşamanın çıktısı ayrı ayrı kontrol edilebilir.
- Çıktı formatını daha tutarlı hale getirir.

Dezavantajları

- Her adım ayrı LLM çağrısı gerektirebilir.
- Token maliyeti ve işlem süresi artabilir.
- İlk adımlarda yapılan hata sonraki adımlara taşınabilir.

- Çok uzun zincirler yönetimi zorlaştırabilir.

AI Engineer Notu

Prompt Chaining, karmaşık bir görevi küçük ve sıralı LLM adımlarına bölerek her adımın çıktısını bir sonraki adıma aktaran workflow tasarım desenidir. Özellikle doküman işleme, rapor hazırlama, özetleme ve yapılandırılmış içerik üretiminde kullanılır.

2. Routing

Routing, kullanıcının isteğini analiz ederek onu en uygun agent'a, modele, tool'a veya workflow'a yönlendiren tasarım desenidir. Routing'in temel görevi **problemi çözmek değildir** — şu soruya cevap verir: *"Bu işi kim veya hangi sistem çözmeli?"*

Örneğin bir AI sisteminde: Coding Agent, Retrieval Agent, Billing Agent, Refund Agent, Browser Agent, Weather Tool, Image Generator gibi uzmanlar bulunabilir. Kullanıcıdan gelen isteğe göre doğru hedef seçilir.

Neden Kullanılır?

Büyük AI sistemlerinde tek bir agent'ın bütün görevleri ve tool'ları yönetmesi verimli değildir. Tek agent'a onlarca tool verildiğinde;

- doğru tool seçimi zorlaşabilir,
- prompt büyüyebilir,
- agent'ın odağı dağılabilir,
- yanlış yönlendirme ihtimali artabilir.

Routing sayesinde her istek yalnızca ilgili uzman sisteme gönderilir.

Nasıl Çalışır?

Kullanıcı: "Python ile bir REST API yaz."

```
Kullanıcı İsteği → Router → "Bu bir kodlama görevi." → Coding Agent → Kod Üretimi
```

Kullanıcı: "Şirketimizin yıllık izin politikası nedir?"

```
Kullanıcı İsteği → Router → "Bu kurumsal bilgi sorusu." → Retrieval Agent / RAG → Dokümanlardan Cevap
```

Kullanıcı: "Bugün İstanbul'da hava nasıl?"

```
Kullanıcı İsteği → Router → Weather Tool → Güncel Hava Verisi
```

Routing Türleri

1. Rule-Based Routing — Karar önceden tanımlanmış kurallarla verilir.

```
Mesaj "fatura" içeriyorsa → Billing Agent
```

`Mesaj "iade" içeriyorsa → Refund Agent`
Avantajı hızlı ve öngörülebilir olmasıdır. Dezavantajı ise farklı ifadeleri anlamakta zorlanmasıdır. Örneğin kullanıcı "*Paramı geri almak istiyorum*." derse, "iade" kelimesi geçmediği için basit kural bunu kaçırabilir.

2. LLM-Based Routing — LLM kullanıcının isteğini analiz eder ve hangi agent'ın uygun olduğuna karar verir.

`"Ürünü geri göndermek istiyorum." → LLM Router → Refund Agent`
Burada model yalnızca kelimelere değil, cümlenin anlamına bakar.

3. Semantic Routing — Cümlelerin anlamsal benzerliğine göre yönlendirme yapar. Örneğin şu cümleler farklı kelimeler içerse de aynı amaca sahiptir:

- "Siparişimi iade etmek istiyorum."
- "Ürünü geri gönderebilir miyim?"
- "Paramı geri almak istiyorum."

Semantic Router bunların hepsini Refund Agent'a yönlendirebilir.

Avantajları

- Doğru görevin doğru uzmana verilmesini sağlar.
- Agent'ların tool sayısını azaltır.
- Sistem daha modüler hale gelir.
- Maliyet ve gecikme azaltılabilir.
- Her agent yalnızca kendi alanına odaklanır.

Dezavantajları

- Yanlış yönlendirme bütün süreci yanlış başlatabilir.
- Çok fazla agent olduğunda seçim zorlaşabilir.
- LLM tabanlı routing ek maliyet oluşturur.
- Router açıklamaları net değilse model yanlış agent'ı seçebilir.

AI Engineer Notu

Routing, kullanıcı isteğini analiz ederek görevi en uygun agent'a, modele, tool'a veya workflow'a yönlendiren tasarım desenidir. Rule-based, LLM-based veya semantic yöntemlerle uygulanabilir.

3. Parallelization

Parallelization, birbirinden bağımsız görevleri sırayla değil, **aynı anda** çalıştırarak toplam işlem süresini azaltan workflow tasarım desenidir.

Kısaca: Birbirini beklemesi gerekmeyen işleri eş zamanlı yürüt.

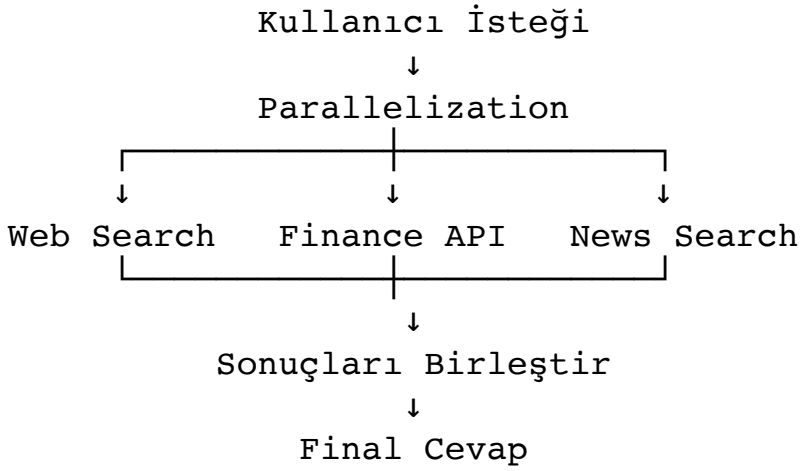
Neden Kullanılır?

Bazı görevlerin sonuçları birbirine bağlı değildir. Örneğin: "*Apple hakkında web araştırması yap, finansal verilerini getir ve güncel haberleri incele*." Bu üç iş birbirinden bağımsızdır.

Sırayla çalıştırılırsa:

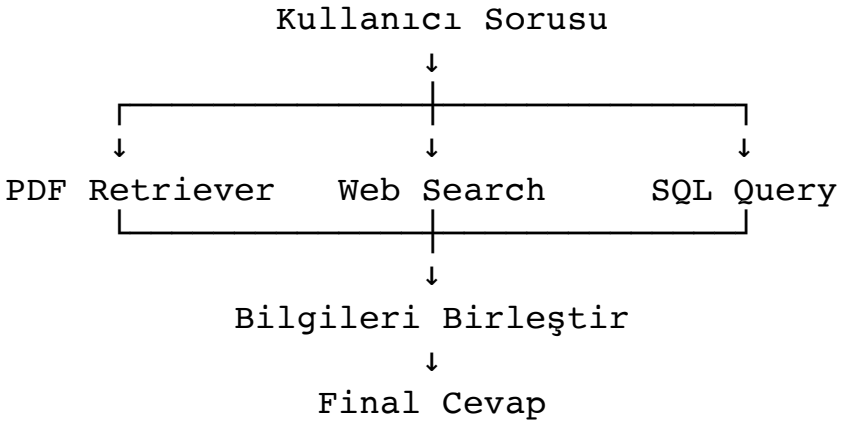
`Web Araştırması: 10 sn → Finans Verisi: 8 sn → Haber Araması: 7 sn`
Toplam yaklaşık 25 saniye sürebilir. Paralel çalıştırılırsa üç görev aynı anda başlar ve toplam süre en uzun görevin süresine yaklaşır.

Nasıl Çalışır?



Gerçek Hayattan Örnek

Kullanıcı: "Bu konu hakkında hem şirket dokümanlarına hem internete hem de veritabanına bak."Sistem aynı anda: Retrieval Agent ile PDF'leri arar, Browser Agent ile web araştırması yapar, SQL Agent ile kurumsal verileri getirir.



Parallelization Türleri

Sectioning — Bir görev farklı alt parçalara bölünür ve her parça aynı anda çalıştırılır. Örneğin bir raporun finans bölümü, pazar analizi bölümü, risk analizi bölümü farklı agent'lar tarafından paralel hazırlanabilir.

Voting — Aynı görev birden fazla modele veya agent'a verilir. Sonuçlar karşılaştırılarak ortak karar üretilir. Örneğin üç agent aynı metni değerlendirir ve çoğunluk kararına göre sonuç seçilir. Bu yaklaşım özellikle güvenlik, sınıflandırma ve kalite kontrol görevlerinde kullanılabilir.

Avantajları

- Toplam işlem süresini azaltır.
- Agent'lar ve tool'lar eş zamanlı çalışabilir.
- Büyük bilgi toplama görevlerini hızlandırır.
- Bağımsız alt görevlerde verimliliği artırır.

Dezavantajları

- Daha fazla CPU, API veya LLM kaynağı tüketebilir.
- Sonuçların birleştirilmesi gerekir.
- Eş zamanlı görevlerden biri başarısız olursa hata yönetimi gerekir.
- Birbirine bağımlı görevlerde kullanılamaz.

AI Engineer Notu

Parallelization, birbirinden bağımsız agent, tool veya LLM görevlerini eş zamanlı çalıştırarak toplam gecikmeyi azaltan workflow tasarım desenidir. Özellikle çoklu veri kaynağı araştırmalarında ve büyük ölçekli analizlerde kullanılır.

4. Orchestrator–Workers

Orchestrator–Workers, büyük ve karmaşık bir görevin bir **Orchestrator** tarafından analiz edilip alt görevlere bölündüğü, bu görevlerin uygun **Worker Agent'lara** dağıtıldığı ve sonuçların yeniden Orchestrator tarafından birleştirildiği workflow tasarım desenidir.

Kısaca: Orchestrator planlar ve dağıtır; Worker'lar yapar; Orchestrator sonuçları birleştirir.

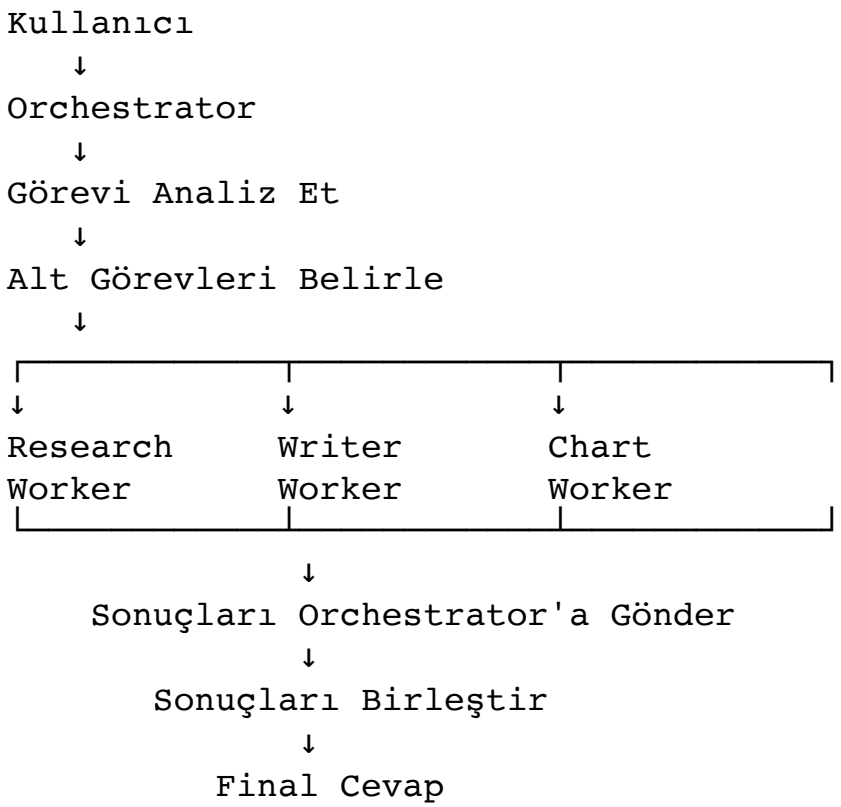
Neden Kullanılır?

Bazı görevlerde alt görevler **önceden kesin olarak bilinmez.** Örneğin: "*Yapay zekâ agent'ları hakkında kapsamlı bir araştırma raporu hazırla*." dendiğinde sistem başlangıçta kaç bölüm gerektiğini, hangi kaynakların inceleneceğini veya hangi uzmanlıkların kullanılacağını dinamik olarak belirlemek zorunda olabilir.

Orchestrator bu noktada;

- görevi analiz eder,
- gerekli alt görevleri oluşturur,
- uygun Worker'ları seçer,
- sonuçları takip eder,
- gerektiğinde ek görev oluşturur,
- final cevabı birleştirir.

Nasıl Çalışır?



Gerçek Hayattan Örnek

Kullanıcı: "Bir şirket için pazar analiz raporu hazırla."

Orchestrator şu alt görevleri oluşturabilir:

- **Research Worker** → Sektör verilerini toplar.
- **Competitor Worker** → Rakipleri analiz eder.
- **Finance Worker** → Finansal verileri inceler.
- **Writer Worker** → Raporu yazar.
- **Chart Worker** → Grafikler oluşturur.

Orchestrator tüm çıktıları bir araya getirerek tutarlı bir rapor üretir.

Orchestrator'ın Görevleri

Orchestrator yalnızca görevi dağıtan basit bir router değildir. Aynı zamanda:

- görevi parçalar,
- bağımlılıkları belirler,
- Worker seçer,
- ilerlemeyi takip eder,
- başarısız görevi yeniden atayabilir,
- eksik bilgi varsa yeni Worker çağırabilir,
- sonuçları sentezler.

Bu nedenle Orchestrator hem planlayıcı hem koordinatör gibi davranabilir.

Avantajları

- Karmaşık ve dinamik görevleri yönetebilir.
- Worker'lar uzmanlık alanlarına göre seçilebilir.
- Görevler gerektiğinde paralel çalıştırılabilir.
- Sistem modüler ve ölçeklenebilir hale gelir.
- Alt görevler başlangıçta sabit olmak zorunda değildir.

Dezavantajları

- Orchestrator darboğaz oluşturabilir.
- Yanlış görev ayrıştırması bütün süreci etkileyebilir.
- Çok sayıda Worker bağlam ve maliyet sorunlarına yol açabilir.
- Sonuçların birleştirilmesi zor olabilir.
- Orchestrator'ın context window'u büyüyebilir.

AI Engineer Notu

Orchestrator–Workers, karmaşık bir görevin Orchestrator tarafından dinamik alt görevlere ayrıldığı, görevlerin uzman Worker'lara dağıtıldığı ve sonuçların tekrar birleştirildiği workflow tasarım desenidir.

5. Evaluator–Optimizer

Nedir?

Evaluator–Optimizer, bir agent'ın veya modelin ürettiği çıktının başka bir agent tarafından değerlendirilmesini ve geri bildirime göre yeniden iyileştirilmesini sağlayan workflow tasarım desenidir.

İki temel rol vardır:

- **Generator / Optimizer:** Çıktıyı üretir veya düzeltir.
- **Evaluator:** Çıktıyı belirli kriterlere göre kontrol eder.

Kısaca: Üret, değerlendir, geri bildirime göre düzelt.

Neden Kullanılır?

LLM'ler ilk denemede her zaman en iyi sonucu üretmez. Çıktı;

- eksik olabilir,
- yanlış bilgi içerebilir,
- kalite kriterlerini karşılamayabilir,
- güvenlik problemi içerebilir,
- istenen formata uymayabilir,
- kod hatası içerebilir.

Bu nedenle çıktı doğrudan kullanıcıya gönderilmeden önce bir değerlendirme aşamasından geçirilebilir. Evaluator açık ve uygulanabilir geri bildirim üretirse, Optimizer bu geri bildirim üzerinden sonucu iyileştirebilir.

Nasıl Çalışır?

Kullanıcı: "Python ile bir REST API yaz."

```
Kullanıcı
   ↓
Generator / Coding Agent
   ↓
İlk Kod
   ↓
Evaluator Agent
   ↓
"Güvenlik kontrolü eksik ve hata yönetimi yok."
   ↓
Optimizer Agent
   ↓
Kodu Düzenle
   ↓
Evaluator Yeniden Kontrol Eder
   ↓
Kriterler Karşılandı mı?
   ├─ Hayır → Yeniden Düzelt
   └─ Evet → Final Kod
```

Bu döngü belirlenen kalite seviyesine ulaşılana veya maksimum tekrar sayısına gelinene kadar devam edebilir.

Gerçek Hayattan Örnek

Bir makale hazırlama sistemi düşünelim.

- **Writer Agent** makalenin ilk taslağını yazar.
- **Evaluator Agent** şu kriterlere göre inceler: Bilgiler doğru mu? Anlatım açık mı? Kaynaklar yeterli mi? Tekrarlar var mı? İstenen format korunmuş mu?
- **Optimizer Agent** geri bildirim doğrultusunda metni düzenler.
- Gerekirse Evaluator tekrar kontrol eder.

Kod Üretiminde Kullanımı

Coding Agent bir özellik geliştirir. Evaluator; testleri çalıştırabilir, statik analiz yapabilir, güvenlik açıklarını arayabilir, gereksinimlerin karşılanıp karşılanmadığını kontrol edebilir. Optimizer ise hataları gidererek kodu yeniden üretir.

Bu yapı Coding Agent'larda oldukça faydalıdır çünkü **"kod üretildi" demek kodun gerçekten çalıştığı anlamına gelmez.**

Evaluator'ın Kriterleri

Evaluator'ın görevi yalnızca "iyi" veya "kötü" demek değildir. İyi bir Evaluator somut geri bildirim vermelidir: Eksik:

```
- Kimlik doğrulama yok.
- Hata durumları ele alınmamış.
- Test kapsamı yetersiz.

Düzeltme önerisi:
- JWT doğrulaması ekle.
- 404 ve 500 hata handler'ları oluştur.
- En az üç birim testi ekle.
```

Bu geri bildirim Optimizer'ın gerçekten düzeltme yapabilmesini sağlar.

Avantajları

- Çıktı kalitesini artırır.
- Hataların kullanıcıya ulaşmasını azaltır.
- Kod, rapor ve içerik üretiminde güvenilirliği yükseltir.
- Sisteme bir kalite kontrol katmanı ekler.
- Kriterlere dayalı iyileştirme yapılmasını sağlar.

Dezavantajları

- Her değerlendirme döngüsü ek LLM çağrısı gerektirir.
- Token maliyeti ve gecikme artar.
- Döngü düzgün sınırlandırılmazsa gereksiz tekrar oluşabilir.
- Evaluator'ın kendisi yanlış değerlendirme yapabilir.
- Belirsiz kalite kriterleri faydasız geri bildirim üretebilir.

AI Engineer Notu: Evaluator–Optimizer, bir Generator'ın ürettiği çıktının ayrı bir Evaluator tarafından kriterlere göre incelendiği ve geri bildirim doğrultusunda tekrar iyileştirildiği workflow tasarım desenidir. Kod, rapor, araştırma ve kritik içerik üretiminde kalite kontrol amacıyla kullanılır.

Workflow'ların Genel Karşılaştırması

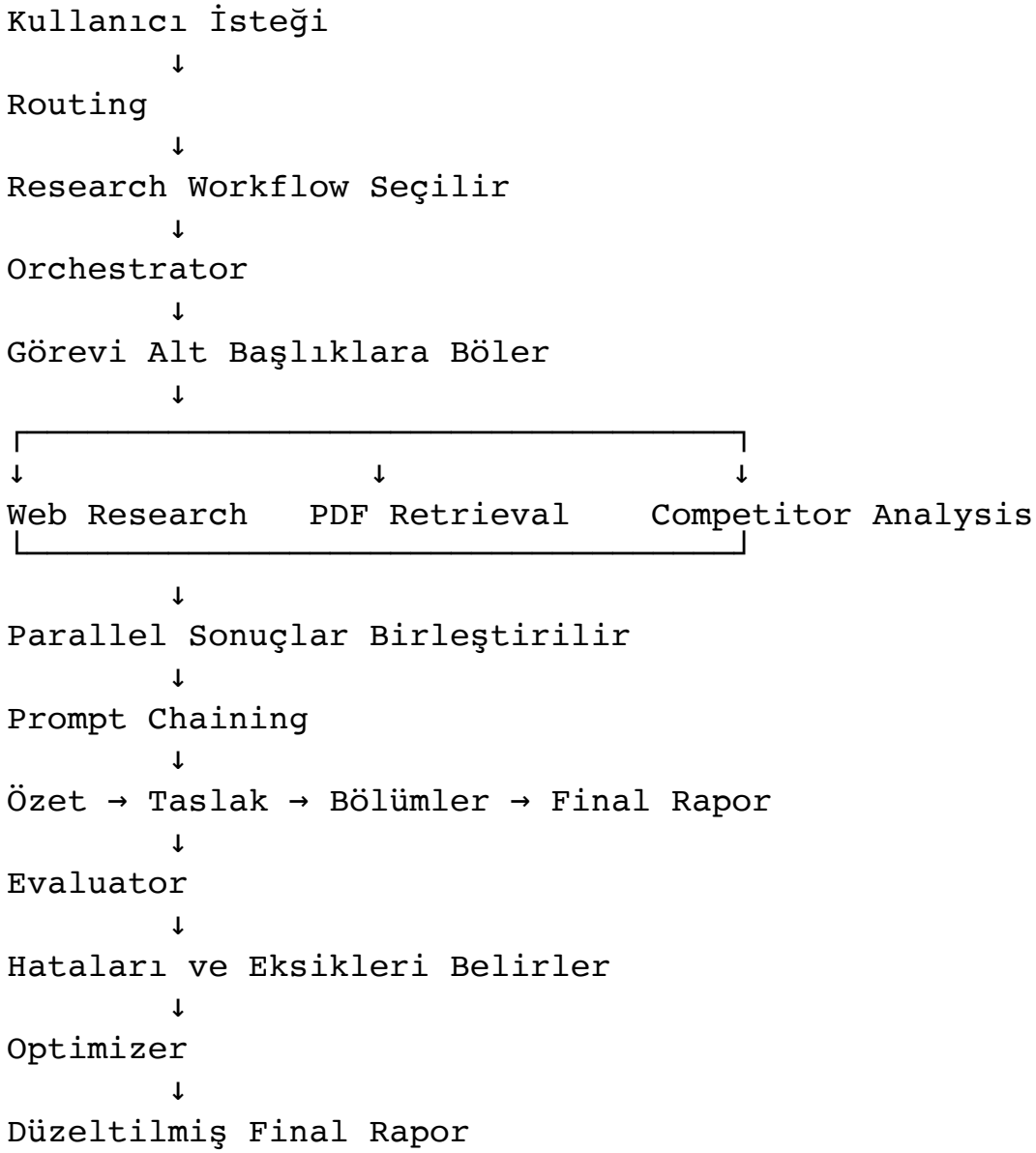
Pattern	Yapı	Ana Fayda	Ne Zaman Kullanılır
Prompt Chaining	Sıralı adımlar	Karmaşık görevi yönetilebilir hale getirir	Doküman işleme, rapor hazırlama, özetleme
Routing	Tek seçim noktası	Doğru uzmana yönlendirme	Çok sayıda farklı görev türü olan sistemler

Parallelization	Eş zamanlı çalışma	Süreyi kısaltır	Bağımsız çoklu veri kaynağı araştırmaları
Orchestrator–Workers	Dinamik görev bölme + toplama	Belirsiz/karmaşık görevleri yönetir	Kapsamlı araştırma ve rapor sistemleri
Evaluator–Optimizer	Döngüsel üret-kontrol et	Kalite kontrolü sağlar	Kod üretimi, kritik içerik üretimi

Workflow'lar Birlikte Nasıl Kullanılır?

Bu desenler birbirinin alternatifi değildir. Gerçek sistemlerde çoğu zaman **birlikte kullanılır**.

Kullanıcı: "Şirketimizin yapay zekâ stratejisi hakkında kapsamlı bir rapor hazırla."



Bu sistemde beş workflow'un tamamı birlikte kullanılmış olur.

Bölümün En Önemli Ayrımı

- **Agent türleri** → Sistemde hangi uzman agent'ların olduğunu açıklar. (örn. *Coding Agent*)
- **Multi-Agent mimarileri** → Agent'ların birbirleriyle nasıl organize olduğunu açıklar. (örn. *Supervisor Pattern*)
- **Workflow design patterns** → Görevin hangi işlem düzeniyle çözüldüğünü açıklar. (örn. *Evaluator–Optimizer*)

Workflow, agent'ın **kim olduğunu** değil; işi **hangi adımlarla ve hangi stratejiyle yaptığını** tanımlar.

7. Agent Framework'ler

Önce Agent Framework Nedir?

Tek başına bir LLM (GPT, Gemini, Claude vb.) sadece soru-cevap verebilir. Ama gerçek hayatta bir AI Agent'ın; tool kullanması, dosya okuması, database'e gitmesi, başka agent'larla konuşması, hafızasının olması, plan yapması, hata durumlarını yönetmesi gerekir.

İşte bütün bunları sıfırdan kodlamak yerine kullandığımız yapılara **Agent Framework** denir.

Kısacası: LLM beyinse, Agent Framework onun iskeleti ve sinir sistemidir.

Frameworklerin Temel Amacı

Aslında hepsi aynı problemi çözmeye çalışır:

```
Kullanıcı
   ↓
 Agent
   ├── Tool çağır
   ├── Hafızayı kullan
   ├── Workflow çalıştır
   ├── Gerekirse başka agent'a devret
   └── Sonucu döndür
```

Ortak kavramlar: **Tool Calling, Memory, Planning, Workflow, Multi-Agent, Tracing.** Fark, bunları *nasıl* yaptıklarıdır.

1) OpenAI Agents SDK

OpenAI'nin kendi geliştirdiği resmi Agent Framework'üdür. GPT modelleriyle agent geliştirmek için hazır bir SDK sunar. Eskiden bunların hepsi elle yazılırken artık hazır geliyor.

- **Tool Calling:** Otomatik yapılır. Örn. "*İstanbul'dan Ankara'ya uçak fiyatı*" → LLM → Tool gerekli → `get_flight_price()` → Sonucu al → Cevap ver.
- **Handoff:** En önemli özelliklerinden biri. Bir agent işi başka bir agent'a devredebilir (örn. Customer Agent → Technical Agent → Billing Agent). Agent kendi karar vererek işi aktarabilir.
- **Guardrails:** Güvenlik katmanıdır. Örn. kullanıcı "Bana kredi kartlarını göster" derse Guardrail bunu engelleyebilir — agent'ın ne yapıp ne yapamayacağını belirler.
- **Tracing:** Agent'ın attığı her adımı görebilirsin (Prompt → LLM → Tool → Tool Sonucu → Final Cevap). Debug yapmayı kolaylaştırır.

Ne zaman kullanılır? OpenAI modelleri kullanıyorsan ve hızlı agent geliştirmek istiyorsan.

2) LangGraph

Tamamen başka bir mantık üzerine kuruludur. Normal frameworklerde A→B→C şeklinde ilerlersin, ama gerçek hayatta işler böyle değildir:

```
Araştır → Yeterli bilgi var mı? → Hayır → Tekrar araştır → Tekrar kontrol et
```

Burada bir döngü oluşur. LangGraph tam bunun için geliştirildi — her işlem bir **node**'dur ve aralarında **edge**'ler bulunur; tam bir graph yapısı oluşur.

- **State Management:** Agent her adımda ne yaptığını unutmaz. Her adımın sonucu state içine yazılır, sonraki node bu state'i okuyabilir.
- **Human in the Loop:** Bazı noktalarda insan onayı isteyebilir (Agent → Karar → İnsan Onayı → Devam). Kurumsal uygulamalarda çok kullanılır.

Ne zaman kullanılır? Uzun süren workflow'larda, RAG sistemlerinde, planning yapan agent'larda.

3) CrewAI

"Neden tek agent olsun? Takım olsun." mantığıyla çalışır. Örn. Research Agent → Writer Agent → Reviewer Agent; her agent'ın görevi farklıdır.

CrewAI buna **Crew** (ekip) der. Her agent'ın **Rolü**, **Hedefi**, **Tool'ları** ayrı ayrı tanımlanır. Örn: Analist araştırıyor → Yazara veriyor → Yazar raporu hazırlıyor → Kontrol agent'ı doğruluyor. Gerçek ekip mantığı.

Ne zaman kullanılır? Araştırma, yazı yazma, rapor oluşturma, içerik üretme.

4) Microsoft AutoGen

CrewAI'de görev paylaşımı vardı; AutoGen'de agent'lar gerçekten birbirleriyle **konuşur**. Örn: Programcı Agent kod yazdı → Reviewer Agent "kod kötü" dedi → Programcı düzeltti → Reviewer onayladı. Tam anlamıyla sohbet ederek çözüm üretirler.

En büyük özelliği: Group Chat — birden fazla agent aynı sohbet içinde fikir alışverişi yapabiliyor.

Microsoft son dönemde AutoGen ve Semantic Kernel'in güçlü yönlerini birleştirerek **Microsoft Agent Framework** altında toplamaya başladı. Yeni projelerde bu birleşik yapıya yöneliyor. AutoGen'i bilmek önemli olsa da uzun vadede Microsoft Agent Framework'ü takip etmek daha doğru olacaktır.

5) Google ADK

Google'ın Gemini modelleri için geliştirdiği kendi Agent Framework'üdür. **Session, Tool, Workflow** hazır olarak gelir. Google Cloud ile çok iyi çalışır (Gemini → BigQuery → Cloud Run → Vertex AI → Agent, hepsi kolayca bağlanabilir).

Kendi CLI araçları, web tabanlı debug ekranı (**ADK Web**) ve yerleşik session yönetimi sunar — geliştirme, test ve dağıtım süreçlerini tek çatı altında toplar.

6) Semantic Kernel

Microsoft'un kurumsal framework'üdür. Amacı AI'yi klasik yazılımlara eklemek. Örn: bir ERP sistemine GPT eklemek istiyorsan Semantic Kernel tam bunun için tasarlanmıştır. İçinde **Plugin, Memory, Planner** vardır. Özellikle .NET tarafında çok güçlüdür.

Ne zaman kullanılır? Enterprise projelerde, kurumsal şirketlerde, Microsoft teknolojileri kullanan firmalarda.

7) Agno (eski adı Phidata)

En büyük amacı: az kod yaz, çabuk bitir. Örn: Tool ekle → Memory ekle → RAG ekle → Bitir. Çok hızlı prototip geliştirilebilir. Multimodal desteği vardır (görsel, ses, metin bir arada kullanılabilir).

Ne zaman kullanılır? Hackathon, demo, PoC, prototip.

8) PydanticAI

En önemli özelliği **Type Safety**. LLM bazen yanlış formatta cevap verir — beklenen `{name, age}` yerine düz metin dönebilir. PydanticAI çıktıyı doğrular (**validation**) ve yanlışsa kabul etmez. Özellikle API geliştiren şirketlerde çok önemlidir.

9) SmolAgents

Hugging Face'in geliştirdiği hafif framework. Çok küçük, kurulumu kolay. En ilginç yanı: **kod yazdırabiliyor.** Örn: "Toplam satışları hesapla" der, Python kodu üretir, kod çalışır, sonuç döner. Bu yüzden **Code Agent** denince akla gelen framework'lerden biridir.

10) LlamaIndex Workflows

LlamaIndex zaten RAG ile ünlüdür. Workflow tarafında **event-driven** mantığını kullanır:

`Event → Workflow → Yeni Event → Başka Workflow`
Belge sistemlerinde çok güçlüdür (PDF → Embedding → Retriever → Agent → LLM, hepsi aynı ekosistemde çalışabilir). Özellikle belge odaklı ve veri yoğun sistemlerde güçlüdür çünkü olay tabanlı akışlarla RAG süreçlerini doğal şekilde yönetir.

Framework	Odak
OpenAI Agents SDK	OpenAI ekosisteminde agent geliştirmek
LangGraph	Karmaşık workflow ve state yönetimi
CrewAI	Birden fazla uzmanın birlikte çalıştığı multi-agent sistemler
Microsoft Agent Framework / AutoGen / Semantic Kernel	Microsoft ve kurumsal projeler
Google ADK	Gemini ve Google Cloud tabanlı uygulamalar
Agno	Hızlı prototip geliştirme
PydanticAI	Çıktının belirli bir şemaya uyması gerektiğinde
SmolAgents	Hafif, kod odaklı agent'lar
LlamaIndex Workflows	RAG ve belge odaklı agent sistemleri

8. Agent Ekosistemindeki Güncel Kavramlar

1. MCP (Model Context Protocol)

Nedir? Model Context Protocol (MCP), AI modellerinin harici sistemlerle (dosya sistemi, veritabanı, API'ler, GitHub, Google Drive, Slack vb.) standart ve güvenli bir şekilde iletişim kurmasını sağlayan açık bir protokoldür.

Eskiden her AI uygulaması için farklı entegrasyon kodları yazılırken, MCP bu problemi ortak bir standart oluşturarak çözer. Böylece MCP destekleyen tüm istemciler (Claude Desktop, ChatGPT vb.) ve MCP Server'lar birbirleriyle uyumlu çalışabilir.

Neden Kullanılır? Bir AI Agent'ın gerçek dünyada işlem yapabilmesi için dış sistemlere erişmesi gerekir. MCP bu erişimi standartlaştırır ve geliştiricilerin her sistem için yeniden entegrasyon yazmasını önler.

Biz de stajda kendi **Leave Manager MCP Server**'ımızı geliştirmiş, **MCP Inspector** ile test etmiş ve **Claude Desktop**'a bağlamıştık.

AI Engineer Notu: MCP, AI Agent'ların harici sistemlerle standart, güvenli ve yeniden kullanılabilir şekilde haberleşmesini sağlayan açık iletişim protokolüdür.

2. Tool Calling

Nedir? Tool Calling, bir LLM'in yalnızca kendi eğitim bilgisiyle cevap vermek yerine gerektiğinde dış araçları (API, veritabanı, Python kodu, web araması vb.) kullanmasını

sağlayan mekanizmadır. Bu sayede model sadece metin üretmez, gerçek dünyada işlem de yapabilir.

Neden Kullanılır? LLM'lerin bilgisi eğitim verileriyle sınırlıdır. Güncel bilgiye erişmek veya işlem yapmak için Tool Calling kullanılır. Örneğin: hava durumunu öğrenmek, SQL sorgusu çalıştırmak, e-posta göndermek, takvime etkinlik eklemek, web araması yapmak, dosya okumak.

Agent mimarisinin temel yapı taşlarından biridir çünkü agent'ın **eylem gerçekleştirebilmesini** sağlar.

AI Engineer Notu: Tool Calling, LLM'in dış araçları kullanarak bilgi almasını veya gerçek dünyada işlem yapmasını sağlayan mekanizmadır.

3. Guardrails

Nedir? Guardrails, AI Agent'ın güvenli, doğru ve belirlenen kurallara uygun şekilde çalışmasını sağlayan kontrol mekanizmalarıdır. Agent'ın yanlış karar vermesini, zararlı içerik üretmesini veya yetkisi olmayan işlemleri gerçekleştirmesini engellemek için kullanılır.

Neden Kullanılır? Bir AI Agent bazen yanlış tool seçebilir, hatalı çıktı üretebilir veya güvenlik açısından risk oluşturabilecek işlemler yapabilir. Guardrails sayesinde;

- kullanıcı girdileri kontrol edilir,
- model çıktıları doğrulanır,
- tool çağrıları sınırlandırılır,
- güvenlik politikaları uygulanır.

Böylece sistem daha güvenilir hale gelir.

AI Engineer Notu: Guardrails, AI Agent'ın güvenli, kontrollü ve belirlenen kurallara uygun çalışmasını sağlayan güvenlik katmanıdır.

4. Memory

Nedir? Memory, AI Agent'ın geçmiş konuşmaları, kullanıcı tercihlerini veya önceki görevleri hatırlayabilmesini sağlayan mekanizmadır. Memory sayesinde agent her istekte sıfırdan başlamaz; geçmiş bilgileri kullanarak daha doğru ve kişiselleştirilmiş kararlar verebilir.

Memory Türleri

- **Short-Term Memory:** Mevcut konuşma veya oturum boyunca kullanılan kısa süreli hafıza.

- **Long-Term Memory:** Kullanıcı tercihleri, geçmiş görevler veya bilgi tabanları gibi uzun süre saklanan bilgiler.

Neden Kullanılır? Memory sayesinde agent; önceki konuşmaları hatırlar, uzun görevleri sürdürebilir, kullanıcı tercihlerini kullanabilir, geçmiş deneyimlerden faydalanabilir. Özellikle Agentic AI sistemlerinde memory, planlama ve karar verme süreçlerinin önemli bir parçasıdır.

AI Engineer Notu: Memory, AI Agent'ın geçmiş bilgileri saklayıp gelecekteki görevlerde kullanmasını sağlayan hafıza mekanizmasıdır.

5. Agent Orchestration

Nedir? Agent Orchestration, tekli veya çoklu agent sistemlerinde tüm agent'ların, tool'ların ve workflow'ların koordineli şekilde yönetilmesini sağlayan katmandır. Sistemin "yöneticisi" gibi çalışır ve hangi agent'ın ne zaman çalışacağına karar verir.

Neden Kullanılır? Karmaşık görevlerde tek bir agent yeterli olmayabilir. Bu durumda orchestration katmanı;

- görevi analiz eder,
- uygun agent'ı seçer,
- tool çağrılarını yönetir,
- workflow'u yürütür,
- gerekirse handoff yapar,
- sonuçları birleştirerek kullanıcıya sunar.

Örneğin bir kullanıcı "*Bana Paris'e uçak bileti bul, hava durumuna bak ve gezi planı hazırla*." dediğinde, orchestration katmanı farklı uzman agent'ları sırayla veya paralel çalıştırarak tüm süreci yönetebilir.

LangGraph, CrewAI, OpenAI Agents SDK ve Google ADK gibi framework'ler farklı seviyelerde orchestration desteği sunar.

AI Engineer Notu: Agent Orchestration, agent'ların, tool'ların ve workflow'ların koordineli şekilde çalışmasını sağlayan yönetim katmanıdır.

Kısa Karşılaştırma

Kavram	Görevi	Sorduğu Soru
MCP	Dış sistemlere standart erişim	"Dış dünyayla nasıl konuşuyorum?"
Tool Calling	Araç kullanarak işlem yapma	"Bu işi hangi araçla yapmalıyım?"
Guardrails	Güvenlik ve kural kontrolü	"Bunu yapmam güvenli mi?"
Memory	Bilgiyi saklama ve hatırlama	"Daha önce ne olmuştu?"
Agent Orchestration	Tüm sistemi koordine etme	"Hangi agent, ne zaman, nasıl çalışmalı?"

Kaynakça

1. **IBM Think** — https://www.ibm.com/think
   (Tool Calling, Agentic Reasoning, Agentic RAG, Planning, Retrieval Agent vb.)
2. **OpenAI** — https://openai.com/
   (Computer Use Agent / Operator)
3. **Ultralytics Documentation** — https://docs.ultralytics.com/
   (Embodied AI)
4. **OpenAI Agents SDK Documentation** — https://openai.github.io/openai-agents-python/
   (Handoffs, Agents as Tools, Agent Frameworkleri)
5. **Anthropic Documentation** — https://docs.anthropic.com/
   (Multi-Agent, Workflow, Agent Design Patterns)
6. **Model Context Protocol (MCP)** — https://modelcontextprotocol.io/
   (MCP)
7. **LangGraph Documentation** — https://langchain-ai.github.io/langgraph/
   (Supervisor Pattern, Workflow, Framework)
8. **LangChain Documentation** — https://python.langchain.com/
   (Workflow ve Agent mimarileri)
9. **CrewAI Documentation** — https://docs.crewai.com/
10. **Microsoft AutoGen Documentation** — https://microsoft.github.io/autogen/
11. **Google Agent Development Kit (ADK)** — https://google.github.io/adk-docs/
12. **Microsoft Semantic Kernel Documentation** — https://learn.microsoft.com/semantic-kernel/
13. **Agno Documentation** — https://docs.agno.com/
14. **PydanticAI Documentation** — https://ai.pydantic.dev/
15. **Hugging Face SmolAgents Documentation** — https://huggingface.co/docs/smolagents/
16. **LlamaIndex Documentation** — https://docs.llamaindex.ai/